

UE2 CHANGES

Laufendes Änderungsprotokoll für Exercise 2 (Build Tooling, TypeScript & CI/CD).

Baut auf dem fertigen Stand von [../ue1/UE1_CHANGES.md](#) auf (ES-Module-Split aus UE1 ist Voraussetzung).

Demo 1 — Package Manager & Projekt-Metadaten

Einfach erklärt — worum geht's hier überhaupt?

Bevor wir an Vite, TypeScript oder automatische Deployment-Skripte (spätere Demos) überhaupt rankönnen, brauchen wir ein **Fundament**: einen Weg, fremden Code sauber ins Projekt zu holen, und eine Art „Ausweis“ für unser eigenes Projekt. Genau das ist Demo 1.

Was ist ein Package Manager? Stell dir vor, du baust ein Möbelstück. Du könntest jede Schraube selbst schmieden — dauert ewig, fehleranfällig. Oder du bestellst fertige Teile aus einem Katalog. Ein **Package Manager** (hier: **npm**) ist genau so ein Bestellsystem für fertigen Programmier-Code („Pakete“/„Bibliotheken“ — Code, den andere geschrieben haben und du einfach benutzt, statt ihn selbst zu schreiben). npm merkt sich dabei genau, welche Version du hast, holt automatisch mit, was ein Paket selbst wieder braucht, und sagt dir, wenn's was Neueres/Sichereres gibt. Reines "Datei runterladen und in Ordner legen" kann das alles nicht — da müsstest du händisch Buch führen.

Warum npm und nicht pnpm? Beide machen im Kern dasselbe. npm ist automatisch da, sobald Node.js installiert ist — kein Extra-Programm nötig. pnpm spart Festplattenplatz, ist aber ein Zusatzwerkzeug, das man separat installieren müsste. Wir nehmen npm, weil du damit schon Erfahrung hast, und wir in dieser Übung eh schon genug Neues lernen (Vite, TypeScript, CI/CD).

Was ist `package.json`? Eine Zutatenliste + Ausweis für dein Projekt, alles in einer Datei. Jedes Werkzeug in der Kette (npm selbst, später Vite, später TypeScript) liest diese eine Datei: wie heißt das Projekt, welche Version, und — am wichtigsten — welche Bausteine braucht es.

Was ist `node_modules/`? Wenn npm ein Paket runterlädt, landet der **tatsächliche Code** dieses Pakets hier — das "physisch ausgepackte" Paket. Kann tausende Dateien sein, lässt sich aber jederzeit exakt neu erzeugen (`npm install`). Deshalb landet es **nie** in Git — wie die leere Versandverpackung, die man nicht für immer aufhebt, wenn man das Produkt jederzeit neu bestellen kann.

Was ist `package-lock.json`? Die wichtigste, am wenigsten offensichtliche Datei. In `package.json` steht nicht "exakt Version 1.11.23", sondern "`^1.11.23`" — das `^` heißt "diese oder auch gern eine neuere, solange sie noch mit `1.` anfängt". Das ist wie ein Rezept, das "füg Mehl hinzu" sagt, ohne Menge/Marke zu nennen. `package-lock.json` ist das Rezept mit **exakten Grammangaben**: genau diese Version, dieser Anbieter, dieser Prüf-Code. Wird sie mit committet, bekommen alle — du, Teamkolleg:innen, später ein automatischer Prüf-Computer (CI) —

garantiert exakt dieselbe Version. Ohne sie: jeder könnte eine leicht andere Version bekommen, je nachdem wann er installiert hat — "bei mir geht's, bei dir nicht", obwohl "derselbe Code" da ist.

Task 1 — npm oder pnpm?

npm. Grund: bereits Erfahrung damit (weniger neue Werkzeuge gleichzeitig lernen, während in dieser Übung schon Vite + TypeScript + GitHub Actions neu dazukommen), und npm ist Teil jeder Node.js-Installation — keine zusätzliche Installation nötig, jeder mit Node kann das Projekt sofort clonen und `npm install` ausführen. pnpm hat reale Vorteile (siehe F4), die rechtfertigen hier aber keinen Umweg.

Task 2 — `package.json` initialisiert und sauber ausgefüllt

Befehl (im Terminal vom Nutzer ausgeführt): `npm init -y`. Das legt ein generisches `package.json` an (Name/Description/Repo-URL zieht npm automatisch aus `README.md` bzw. dem Git-Remote — daher stimmen die von Anfang an).

Danach von Hand nachgeschärft, Feld für Feld:

Feld	npm-Default	Geändert zu	Warum
<code>version</code>	<code>1.0.0</code>	<code>0.1.0</code>	SemVer: <code>0.x</code> heißt „noch keine stabile, versprochene API/Release" — passt zu einem Coursework-Projekt, das gerade erst tooling-mäßig aufgebaut wird. <code>1.0.0</code> würde eine Stabilitätszusage vortäuschen, die es nicht gibt.
<code>private</code>	<i>(fehlt)</i>	<code>true</code>	Verhindert aus Versehen <code>npm publish</code> auf das öffentliche npm-Registry — dieses Projekt ist eine App, kein zu veröffentlichendes Paket.

Feld	npm-Default	Geändert zu	Warum
<code>type</code>	<code>"commonjs"</code>	<code>"module"</code>	Der komplette Code läuft seit UE1 schon als native ES-Module (<code>import/export</code>). <code>"type": "module"</code> sagt Node dasselbe für alle <code>.js</code>-Dateien im Projekt (z. B. spätere Config-Dateien wie <code>vite.config.js</code>) — vermeidet CommonJS/ESM-Reibung, bevor sie in Demo 2 überhaupt auftreten kann.
<code>main</code>	<code>"app.js"</code>	<i>entfernt</i>	<code>main</code> ist der Einstiegspunkt für Pakete, die andere per <code>require/import "dieses-paket"</code> einbinden. Diese App wird nie als npm-Paket importiert (sie hat <code>index.html</code> als echten Einstiegspunkt, nicht <code>main</code>) — und <code>"app.js"</code> zeigte ohnehin auf die alte Vor-Refactor-Datei aus UE1, die wir nur zum Diffen behalten haben. Irreführend, also raus.
<code>license</code>	<code>"ISC"</code>	<code>"UNLICENSED"</code>	Kein Open-Source-Release, kein Interesse an Weiterverwendung durch Dritte. <code>"UNLICENSED"</code> + <code>private: true</code> ist die npm-Konvention dafür.

Feld	npm-Default	Geändert zu	Warum
<code>scripts</code>	<pre>{ "test": "echo \\\"Error...\\\" && exit 1" }</pre>	<pre>{}</pre>	Der Default-Test-Script ist ein Fake (er tut nichts außer mit Fehlercode abzubrechen) — irreführend, bis es echte Scripts gibt. Echte Scripts (<code>dev</code> , <code>build</code> , <code>lint</code> , <code>lint:fix</code> , <code>format</code>) kommen ab Demo 2/4.
<code>author</code>	<code>""</code>	<code>"Patrick Lukesch"</code>	Leeres Feld ausgefüllt.
<code>keywords</code>	<code>[]</code>	<code>["case-study", "vanilla-js", "web-engineering-course"]</code>	Kurz beschreibend; für ein privates Projekt nicht funktional wichtig, aber „richtig ausgefüllt“ statt leer.
<code>description</code> , <code>repository</code> , <code>bugs</code> , <code>homepage</code>	von npm automatisch gesetzt	unverändert (<code>description</code> leicht erweitert)	Waren schon korrekt (aus <code>README.md</code> /Git-Remote abgeleitet).

Task 3 — `.gitignore`

Ergänzt (`.gitignore`, neuer Abschnitt „Node / npm“):

```
node_modules/
dist/          # vite-build-output, kommt in demo 3 - schon vorausschauend drin
.env
.env.local
```

`node_modules/` ist Pflicht (siehe F1). `dist/` schon jetzt mit rein, obwohl es erst in Demo 3 entsteht — spart eine zweite `.gitignore`-Änderung später. `.env*` als Vorgriff auf mögliche Secrets (aktuell ungenutzt, aber Standard-Praxis, bevor man sie braucht statt danach).

Task 4 — eine echte Abhängigkeit installiert

Befehl: `npm install dayjs`. Gewählt statt eines Wegwerf-Beispiels, weil die App mit `formatDate()` in `js/utils.js` schon echten Datums-Formatierungscode hat — `dayjs` ist also eine Abhängigkeit mit echtem Bezug zur App, kein Selbstzweck. (Vite selbst installieren wir bewusst *nicht* hier, sondern erst in Demo 2 — dort gehört es laut Angabe hin.)

Ergebnis in `package-lock.json` (`lockfileversion: 3`):

```
"node_modules/dayjs": {
  "version": "1.11.23",
  "resolved": "https://registry.npmjs.org/dayjs/-/dayjs-1.11.23.tgz",
  "integrity": "sha512-QDTCUOM0MxR3hQfn1DJfwekQiaanm1ubOD231u73WBckQ/fsamwRLiE2GBz6D3a/xF1NgfiDLJjXBalHYOYTtQ==",
  "license": "MIT"
}
```

dayjs hat **null eigene Abhängigkeiten** (nur ein einziger `node_modules/...`-Eintrag neben dem Wurzelpaket) — bewusst simpel für den Einstieg, aber die Lockfile-Mechanik (exakte Version + Integrity-Hash) gilt genauso für Pakete mit hundert transitiven Abhängigkeiten.

Verifikation

`package.json` und `package-lock.json` gegengelesen: `dependencies.dayjs` in beiden konsistent (`^1.11.23` in `package.json`, aufgelöst auf exakt `1.11.23` im Lockfile). `node_modules/` + `package-lock.json` + `package.json` waren vor der `.gitignore`-Änderung alle als „untracked“ gelistet (`git status`) — nach der Änderung verschwindet nur `node_modules/` aus der Liste, `package.json` / `package-lock.json` bleiben sichtbar zum Committen (genau das will die Aufgabe: Lockfile **committet**, `node_modules` **nicht**).

Live-Demo — was du im Unterricht herzeigst

Reihenfolge, mit der du 10–15 Min füllst, ohne dich zu verhaspeln:

1. `package.json` **im Editor aufmachen**. Zeig jedes Feld, sag in einem Satz wofür (Zutatenliste-Analogie). Zeig gezielt `private: true`, `type: "module"`, `license: "UNLICENSED"` — das sind die drei, die npm **nicht** automatisch richtig gesetzt hat.
2. **Terminal**: `npm init -y` kurz erklären (nicht nochmal ausführen, nur den Screenshot/die Erinnerung zeigen) — "hat mir eine Standard-`package.json` gebaut, Name/Beschreibung/Repo-Link hat npm sich automatisch aus `README.md` und dem Git-Remote gezogen."
3. `package-lock.json` **aufmachen**, zum `"node_modules/dayjs"`-Eintrag scrollen. Zeig `version`, `resolved` (die genaue Download-URL), `integrity` (der Prüf-Hash) — sag: "das ist das Rezept mit exakten Grammangaben."
4. **Terminal, live**: `git status` — zeig, dass `node_modules/` **nicht** auftaucht (dank `.gitignore`), aber `package.json` / `package-lock.json` schon. Dann optional im Explorer/VS Code den `node_modules`-Ordner aufklappen — "hunderte Dateien, alle reproduzierbar, alle bewusst nicht im Repo."
5. **GitHub im Browser öffnen** (euer Repo, „Commits“-Historie), den Demo-1-Commit anklicken → zeigt den Diff von `package.json` / `.gitignore` farbig (rot = vorher, grün = nachher) — sehr überzeugend fürs Publikum, weil man Vorher/Nachher in einem Bild sieht.

DevTools brauchst du für Demo 1 nicht — kommt erst ab Demo 2.

Demo 2 — Vite als Dev-Server

Einfach erklärt — worum geht's hier überhaupt?

In UE1 lief ein ganz simpler "Briefträger" (`python -m http.server`): ein Programm, das nur eine Sache kann — "du fragst nach einer Datei, ich geb sie dir, unverändert." Mehr nicht.

Vite ist auch so ein Briefträger-Programm — zeigt dir deine Webseite im Browser, während du programmierst. Aber viel schlauer: es merkt automatisch, wenn du eine Datei änderst, und schickt

die Änderung **live** an den Browser (das ist HMR, dazu unten mehr). Später (Demo 3) kann es deine ganze Webseite außerdem für den echten, öffentlichen Einsatz "verpacken".

Warum mussten wir Ordner verschieben (`data/`, `assets/` → `public/`)? Vite schaut sich `index.html` an und verfolgt automatisch alles, was dort direkt verlinkt ist. Was **erst zur Laufzeit** im JavaScript nachgeladen wird (unser `fetch("data/case.json")`, oder Bilder, die erst beim Rendern aus JSON-Daten zusammengebaut werden) — das sieht Vite **im Voraus nicht**. Der Ordner `public/` ist Vites Sonderregel dafür: alles darin liefert Vite **einfach unverändert aus**, egal ob es's "sieht" oder nicht — wie ein Zimmerservice, der alles auf den Tresen im Foyer stellt, egal wer wann danach fragt. Weil `public/`-Inhalte unter demselben Pfad landen wie vorher, musste im Code selbst **nichts** geändert werden.

Was ist `vite.config.js`? Eine Einstellungs-Datei fürs Vite-Programm, so wie die "Einstellungen"-Seite an einem Handy. Unsere ist fast leer, weil Vite von sich aus schon richtig rät, was wir brauchen.

Was hat `npm install -D vite` gemacht? Dasselbe Prinzip wie bei `dayjs` in Demo 1 — Vite wird heruntergeladen und lokal abgelegt. Das `-D` heißt "ich brauche das nur beim Entwickeln, nie im Browser der späteren Besucher:innen" (siehe Demo-1-Frage 2).

Was macht `npm run dev`? Startet Vite tatsächlich — wie ein Doppelklick auf ein Programm-Icon. Ab dann läuft die App unter `http://localhost:5173`.

Was ist HMR? Hot Module Replacement — auf Deutsch grob "heißer Teil-Austausch". „Hot“ = während das Programm noch **läuft** (nicht aus- und wieder einschalten). „Module“ = ein einzelner Baustein deines Codes. „Replacement“ = austauschen. Also: **während die Seite läuft, wird ein einzelner Baustein ausgetauscht, ohne alles andere anzufassen**.

Analogie: Du liest ein Buch, bist auf Seite 150. Jemand tauscht nur Seite 47 aus (ein Tippfehler wurde korrigiert), du liest einfach auf Seite 150 weiter. Das ist HMR. Ein **komplettes Neuladen** (Browser-F5) wäre dagegen: jemand nimmt dir das ganze Buch weg, klappt es

zu, du musst wieder bei Seite 1 anfangen — auch wenn sich nur eine Seite geändert hat. Du verlierst deine Stelle, dein Lesezeichen, alles.

Wir haben beide Fälle getestet: **CSS-Änderung** → nur die eine "Seite" (das Stylesheet) wird getauscht, alles bleibt erhalten. **JS-Änderung** → bei uns (weil unsere Module keinen Extra-Code dafür haben) macht Vite lieber einen sauberen Komplet-Neustart, als zu riskieren, den Programm-Zustand kaputt zu machen — Zustand geht dabei verloren.

Task 1 — Vite installiert + Projekt umstrukturiert

Befehl (Nutzer): `npm install -D vite` → `devDependencies.vite: "^8.3.0"`. `-D`, nicht `-S`/ohne Flag: Vite läuft nur beim Entwickeln/Bauen, ihr eigener Code landet nie im ausgelieferten Bundle (siehe Demo-1-Frage 2 zu `dependencies` / `devDependencies`).

Umstrukturierung — neuer Ordner `public/`, `data/` und `assets/` reingeschoben (per `git mv`, Historie bleibt):

```
public/
├─ data/          (case.json, evidence.json, locations.json, people.json,
timeline.json)
└─ assets/        (logo/logo.svg, people/*.png)
```

`resources/` (die alten, nie referenzierten Doppel-Bilder aus der Zeit vor UE1) bleibt bewusst **außerhalb** von `public/` und außerhalb des Repos-Layouts, das Vite kennen muss — nichts im Code referenziert diesen Ordner, also muss ihn auch niemand "finden". Weiterhin unbenutzter Alt-Ballast, kein Teil dieser Übung.

Warum `public/` und keine Code-Änderung nötig war: `public/` ist Vites Konvention für Dateien, die **unverändert, ungehasht, unter genau demselben absoluten Pfad** ausgeliefert werden — nicht Teil von Vites Modul-Graph/Bundling. Das passt exakt auf unseren Fall:

`fetch("data/case.json")` in `js/data.js` ist ein zur Build-Zeit nicht analysierbarer String, `` für Personen-Avatare wird erst zur Laufzeit aus `people.json` zusammengebaut — beides kann Vite beim Build nicht "sehen" und würde es sonst (ohne `public/`) beim `vite build` in Demo 3 stillschweigend **nicht** mit ausliefern. Weil `public/`-Inhalte 1:1 unter demselben Pfad landen, funktionieren `fetch("data/case.json")` und `` unverändert weiter — **kein einziges Byte** in `index.html` oder `js/` musste angefasst werden.

`vite.config.js` neu angelegt — fast leer (`defineConfig({})`), weil unser Layout (`index.html` im Projekt-Root, `public/` unter Standardnamen) schon exakt zu Vites Zero-Config-Vorgaben passt. Existiert trotzdem schon jetzt als fester Platz für spätere Config (z. B. `base` für GitHub Pages in Demo 9).

`package.json`: `"dev": "vite"` als Script ergänzt.

Task 2 — Dev-Server läuft, jede View geprüft

Befehl (Nutzer): `npm run dev` → Vite startet auf `http://localhost:5173`. Im Browser durchgecheckt:

- Dashboard (`18/6/6/0/1`-Stats), Evidence (18 Karten), People (6 Karten, Avatar-Bild lädt aus `public/assets/`, `naturalwidth: 64` bestätigt „ist wirklich ein geladenes Bild, kein kaputter Link“), Timeline (15 Events, kein `[object object]`), Workspace (Hypothesen-Dropdowns gefüllt) — alle funktionieren identisch zum alten `python -m http.server`-Setup aus UE1.
- Netzwerk-Log zeigt: `/data/*.json` und `/assets/**` werden exakt unter denselben Pfaden wie vorher ausgeliefert (aus `public/`), **jede** `js/*.js`-Datei wird **einzeln** angefragt (kein Bundling im Dev-Modus — natives ESM direkt vom Browser geladen), zusätzlich `/@vite/client` und `/node_modules/vite/dist/client/env.mjs` — beides Dateien, die **nicht** in unserem Quellcode existieren, sondern von Vite selbst on-the-fly in `index.html` injiziert bzw. aus `node_modules` ausgeliefert werden.
- Konsole sauber bis auf `[vite] connecting... / connected.` — Vites eigener HMR-Client baut eine WebSocket-Verbindung zum Dev-Server auf.

Task 3 — HMR ausgelöst und beobachtet

Versuch 1 — CSS-Änderung (`styles.css`, `--color-accent` testweise auf `#e0392b`, danach zurückgesetzt): Konsole zeigt `[vite] css hot updated: /styles.css`, Netzwerk zeigt einen einzelnen neuen Request `GET /styles.css?t=<timestamp>` (Cache-Busting-Query, kein Navigations-Request). Dabei geprüft, was **nicht** passiert ist: eine vorher gesetzte globale JS-Variable (`window.__hmrMarker`) war danach **immer noch da**, `state.bookmarks` unverändert, aktueller Hash unverändert — der komplette JS-Ausführungskontext blieb am Leben, nur die Stylesheet-Regel wurde live ausgetauscht.

Versuch 2 — JS-Modul-Änderung (`js/views/dashboard.js`, ein Kommentar ergänzt, danach zurückgesetzt): dieses Mal loggt die Konsole erneut `[vite] connecting... / connected.` — der Vite-Client baut seine WebSocket-Verbindung **neu** auf, was nur bei einem echten Full-Reload passiert. Bestätigt: die zuvor gesetzte `window.__hmrMarker2`-Variable war **weg** (`undefined`), und ein zuvor geöffnetes Beweisstück-Detail (nicht-persistenter State, `state.selectedEvidence`) war **wieder geschlossen** — der komplette Zustand ist bei diesem Reload verloren gegangen (der Hash in der URL blieb zwar erhalten, weil er Teil der Adresse ist, aber alles, was nur im JS-Speicher lebte, ist weg).

Fazit aus dem Kontrast: unsere Module haben keinerlei `import.meta.hot.accept()`-Code — deshalb fällt Vite bei JS-Änderungen auf einen Full-Reload zurück. CSS-HMR bekommt man bei Vite dagegen automatisch, ganz ohne eigenen Opt-in-Code.

Verifikation

Voller Feature-Durchlauf über den Vite-Dev-Server (nicht mehr `python -m http.server`): alle 5 Views, Konsole sauber, Netzwerk-Log zeigt öffentliche Assets korrekt ausgeliefert. Beide HMR-Teständerungen im Nachhinein vollständig zurückgesetzt (`git diff` auf `styles.css / js/views/dashboard.js` leer) — bleiben nicht im Commit.

Live-Demo — was du im Unterricht herzeigst

Vorbereitung: zwei Fenster nebeneinander — Editor (VS Code) links, Browser rechts, im Browser **DevTools offen** (`F12`).

1. Server starten, App zeigen (Task 1+2)

```
npm run dev
```

Zeig kurz die bunte Vite-Startmeldung im Terminal (Version, "ready in X ms", die `Local`-URL). Öffne `http://localhost:5173` im Browser, klick einmal durch alle 5 Views (Dashboard → Evidence → People → Timeline → Workspace) — "läuft identisch wie vorher mit dem Python-Server."

2. DevTools → Network-Tab, den Unterschied zu einem simplen Server zeigen

- `F12` → Tab **Network** → Seite neu laden (`Strg+R`).
- Zeig in der Liste: `/@vite/client` und `/node_modules/vite/dist/client/env.mjs` — Dateien, die es in `js/ / index.html` **gar nicht gibt**. Sag: "das injiziert Vite selbst, ein normaler Dateiserver könnte das nicht."
- Zeig: jede `js/*.js`-Datei einzeln als eigener Request (kein Bundle) — "Vite liefert jedes Modul einzeln aus, genau weil unser Code seit UE1 in echte ES-Module gesplittet ist."

- Zeig: `data/*.json` und `assets/*.png` laden ganz normal, exakt wie vorher — "aus dem neuen `public/`-Ordner, ohne dass ich am Code was ändern musste."

3. DevTools → Console-Tab

- Zeig `[vite] connecting...` / `connected.` — "das ist Vites eigener Verbindungsaufbau für Live-Updates."

4. HMR live vorführen — CSS (der Beweis, dass NICHTS neu lädt)

- Im Editor: `styles.css` öffnen, `--color-accent: #2f6fed;` kurz auf eine andere Farbe ändern (z. B. `#e0392b`), speichern.
- Im Browser: die Akzentfarbe (Buttons, aktiver Nav-Reiter) ändert sich **sofort**, ohne dass die Seite flackert/neu lädt.
- DevTools-Console zeigt `[vite] css hot updated: /styles.css`.
- DevTools-Network zeigt einen **neuen** Request `styles.css?t=...`, aber **keinen** neuen Dokument-Request (kein weißer Blitz, kein "Navigation" in der Waterfall-Ansicht).
- **Zustand als Beweis:** vorher in der Evidence-Liste ein Beweisstück bookmarken (Stern anklicken) — nach der CSS-Änderung ist der Stern **immer noch aktiv**. Farbe zurücksetzen, wieder speichern.

5. HMR live vorführen — JS (der Kontrast: Full-Reload)

- Im Browser: ein Beweisstück öffnen (Detail-Ansicht), damit sichtbarer, nicht-gespeicherter Zustand existiert.
- Im Editor: `js/views/dashboard.js` öffnen, irgendwo einen harmlosen Kommentar hinzufügen, speichern.
- Im Browser: diesmal lädt die **ganze Seite neu** (kurzes Aufblitzen/Reset), das offene Beweisstück-Detail ist wieder zu.
- DevTools-Console zeigt **erneut** `[vite] connecting...` / `connected.` — "der Client baut seine Verbindung neu auf, das passiert nur bei einem echten Reload."
- Kommentar wieder entfernen.

6. Eine Zeile für den Vergleich, laut aussprechen: "CSS-Änderungen tauscht Vite live aus, ganz automatisch. JavaScript-Änderungen nicht — weil unser Code keinen speziellen Zusatz-Code dafür hat, macht Vite lieber sauber neu, statt den Zustand zu riskieren."

Demo 3 — Produktions-Build & Preview

Einfach erklärt — worum geht's hier überhaupt?

Zuerst die Grundfrage: was IST `dist/` eigentlich, und hab ich jetzt zwei Projekte?

Nein — du hast weiterhin nur EIN Projekt. `dist/` ist **kein zweites Projekt, das du pflegst**, sondern nur eine **Kopie/ein Export**, den ein Programm (Vite) automatisch für dich aus deinem echten Projekt herstellt.

Die Analogie, die es hoffentlich sofort klarmacht: Stell dir vor, du schreibst einen Text in Word. Das Word-Dokument (`.docx`) ist dein **echtes** Projekt — daran arbeitest du, das speicherst du, das ist in Git. Wenn du den Text jemandem schicken willst, klickst du "Als PDF exportieren". Das PDF ist eine **neue, zusätzliche Datei**, aus dem Word-Dokument heraus erzeugt. Du bearbeitest

das PDF **nicht direkt** — willst du was ändern, gehst du zurück ins Word-Dokument, änderst dort, und exportierst ein **neues** PDF (das alte überschreibst du damit).

Genau das ist `dist/`: das "PDF" deines Projekts. Dein **echtes** Projekt bleibt `js/`, `index.html`, `styles.css`, `public/` — das bearbeitest du, das ist in Git. `dist/` wird bei jedem `npm run build` **komplett neu erzeugt** (alter Inhalt wird dabei gelöscht und neu geschrieben). Du fasst nie eine Datei in `dist/` von Hand an — genau wie du ein exportiertes PDF nicht in Word zurückverwandelst und weiterbearbeitest.

Das erklärt auch, warum `dist/` (genau wie `node_modules/` aus Demo 1) in `.gitignore` steht und **nicht** committet wird: es ist jederzeit reproduzierbar — jeder, der deinen echten Quellcode hat, kann sich mit `npm run build` sein eigenes, identisches `dist/` erzeugen. Es mitzuschleppen wäre wie das PDF UND das Word-Dokument UND alle alten PDF-Versionen gemeinsam aufzuheben, obwohl eins davon (Word) reicht, um jedes PDF jederzeit neu zu erzeugen.

Wofür braucht man `dist/` dann überhaupt? Weil dein echter Quellcode (so wie er in `js/` liegt: 13 einzelne, gut lesbare Dateien mit Kommentaren) genau das Richtige zum **Programmieren**

ist, aber **nicht** das, was du später auf einen echten Server hochladen willst (zu viele einzelne Dateien, unnötig groß, mit internen Kommentaren, die niemanden außer dich was angehen).

`dist/` ist die kompakte, fertig verschnürte Version davon — die, die wir in Demo 9 tatsächlich auf GitHub Pages hochladen werden.

Warum waren es "13 Dateien" — welche 13 sind gemeint?

Erinnerung an UE1, Demo 1: dort haben wir die eine riesige `app.js` in viele kleine, themenbezogene Dateien aufgeteilt, die sich per `import / export` gegenseitig benutzen. Das sind genau diese 13 Dateien im Ordner `js/`:

<code>js/main.js</code>	<code>js/navigation.js</code>	<code>js/data.js</code>
<code>js/storage.js</code>	<code>js/state.js</code>	<code>js/utils.js</code>
<code>js/dropdowns.js</code>	<code>js/lookup.js</code>	
<code>js/views/dashboard.js</code>	<code>js/views/evidence.js</code>	<code>js/views/people.js</code>
<code>js/views/timeline.js</code>	<code>js/views/workspace.js</code>	

Im **Dev-Modus** (Demo 2, `npm run dev`) lädt der Browser jede davon **einzeln** über eine eigene Netzwerk-Anfrage — praktisch zum Programmieren, weil Vite bei einer Änderung genau weiß, welche

eine Datei es neu schicken muss (das war die Grundlage für HMR). Beim **Produktions-Build** (diese Demo) braucht dieser Vorteil aber niemand mehr — im Gegenteil, 13 einzelne Netzwerk-Anfragen sind für einen echten Besuch nur unnötig langsam. Deshalb klebt Vite sie beim Bauen zu einer Datei zusammen.

(Nebenbei: die Terminal-Ausgabe von `vite build` sagt "17 modules transformed" — das sind unsere 13 `.js`-Dateien plus `styles.css` plus ein paar kleine interne Helfer-Module, die Vite selbst automatisch dazupackt. Die genaue Zahl 17 musst du dir nicht merken, wichtig ist nur: 13 eigene Quelldateien werden zu 1 Datei.)

Bisher (Demo 2) haben wir Vite nur als "schlaue Briefträger fürs Programmieren" benutzt — der **Dev-Server**. Der ist super zum Entwickeln (schnell, HMR, zeigt Fehler ausführlich), aber **nie dafür gedacht, dass echte Besucher:innen deiner Webseite ihn benutzen**. Demo 3 ist der Schritt "jetzt bauen wir die Version, die wirklich rausgeht".

Analogie: eine Baustelle vs. das fertige, eröffnete Gebäude. Auf der Baustelle (Dev-Server) liegt Werkzeug rum, es ist laut, Handwerker laufen durch, aber genau das macht flexibles Bauen möglich. Wenn das Gebäude fertig ist (Produktions-Build), wird aufgeräumt, alles verpackt und zugeschlossen — Besucher:innen sollen nur das fertige, saubere Ergebnis sehen, nicht die Baustelle.

Was heißt „bauen“ (`vite build`) hier konkret? Vite nimmt unsere 13 einzelnen `js/*.js`-Dateien (die man einzeln über 13 Netzwerk-Anfragen laden müsste) und macht daraus **eine einzige** Datei. Das nennt man **Bundling** — "viele kleine Briefe" werden zu "einem großen Brief", weil ein Postweg schneller ist als dreizehn einzelne.

Zusätzlich wird der Code **minifiziert** — alles rausgeworfen, was nur für *Menschen* beim Lesen wichtig ist (Kommentare, sprechende Variablennamen wie `state` oder `caseRes`, Zeilenumbrüche, Einrückungen), weil der **Browser** das alles gar nicht braucht. Ergebnis bei uns: aus 51 KB lesbarem Code wurden 23 KB — mehr als halbiert, **ohne** dass sich am Verhalten irgendwas ändert.

Warum bekommen die neuen Dateien komische Namen wie `index-B1hg0Ru0.js`? Das ist ein **Content-Hash** — eine kurze, aus dem exakten Dateiinhalt berechnete Zeichenfolge, die sich **automatisch ändert**, sobald sich der Inhalt der Datei auch nur ein bisschen ändert.

Analogie: ein Mindesthaltbarkeitsdatum/Chargen-Code auf einer Lebensmittelverpackung. Ändert sich das Rezept, bekommt die neue Charge eine neue Nummer — niemand verwechselt alte und neue

Charge. Genauso beim Browser: er darf `index-B1hg0Ru0.js` für **immer** in seinem Zwischenspeicher (Cache) behalten, weil dieser exakte Dateiname garantiert *nie* seinen Inhalt ändert — ändert sich der Code, bekommt die Datei automatisch einen neuen Namen, und der Browser

lädt die zwangsläufig neu. Ohne Hash müsste der Browser bei jedem Besuch unsicher nachfragen "hat

sich `main.js` vielleicht geändert?" — mit Hash weiß er es einfach am Namen.

Was ist `vite preview`? Ein ganz simpler Server (ähnlich dem `python -m http.server` aus UE1!), der **nur** das ausliefert, was tatsächlich in `dist/` liegt — kein Bauen, kein HMR, keine Extras. Damit testest du: "funktioniert wirklich die Version, die ich später hochladen würde?" — nicht "funktioniert meine Entwicklungsumgebung".

Task 1 — Build ausgeführt, `dist/` inspiziert

Befehl (Nutzer): `npm run build` → `vite build`.

```
vite v8.3.0 building client environment for production...
✓ 17 modules transformed.
dist/index.html          10.83 kB | gzip: 2.80 kB
dist/assets/index-ZAWMSz9M.css 11.44 kB | gzip: 2.77 kB
dist/assets/index-B1hg0Ru0.js 23.26 kB | gzip: 6.17 kB
✓ built in 136ms
```

Inhalt von `dist/` danach:

```
dist/
├─ index.html
├─ assets/
│   └─ index-B1hg0Ru0.js      <- gebündelt + minifiziert, unsere 13 js/*.js-
Dateien
│   └─ index-ZAWMSz9M.css     <- gebündelt + minifiziert, styles.css
│   └─ logo/logo.svg         <- 1:1 kopiert aus public/assets/
│   └─ people/*.png (6 Stück) <- 1:1 kopiert aus public/assets/
└─ data/*.json (5 Stück)      <- 1:1 kopiert aus public/data/
```

Bemerkenswert: `dist/assets/` enthält **sowohl** die neuen gehashten Bundle-Dateien **als auch** die unveränderten Bilder aus `public/assets/` — reiner Zufall, dass beide „assets“ heißen (Vites Standardordner für Build-Output trägt zufällig denselben Namen wie unser `public/assets/`), aber kein Konflikt, weil die Dateinamen sich nie überschneiden.

Task 2 — mit `vite preview` serviert, End-to-End geprüft

Befehl (Nutzer): `npm run preview` → Server auf `http://localhost:4173`, liefert **ausschließlich** `dist/` aus — kein Vite-Dev-Server, keine Quell-Dateien mehr im Zugriff.

Im Browser geprüft: alle 5 Views identisch (18/6/6/0/1-Stats), Personen-Avatare laden, Timeline ohne Fehler. Zusätzlich ein **echter End-to-End-Test** über die gebaute Version: ein Beweisstück bookmarken → `localStorage`-Eintrag `remotion_bookmarks` enthält danach `["E14"]` — bestätigt, dass nicht nur das Anzeigen, sondern auch die Interaktivität (Klick-Handler, `localStorage`) im gebauten, minifizierten Code noch korrekt funktioniert.

Netzwerk-Log: nur noch **drei** eigene Kern-Requests (`index-B1hg0Ru0.js`, `index-ZAWMSz9M.css`, `logo.svg`) statt der zwölf einzelnen Modul-Requests im Dev-Modus — plus `data/*.json` und `assets/people/*.png` unverändert aus `public/`. **Kein** `/@vite/client`, **kein** `/node_modules/...`-Request mehr — die ganze Dev-/HMR-Maschinerie ist komplett weg. Konsole: komplett leer (kein `[vite] connecting...` mehr — das gibt's nur im Dev-Server).

Task 3 — Dev-Quelle vs. gebautes Ergebnis verglichen

`js/*.js` (Quelle, 13 Dateien) vs. `dist/assets/index-B1hg0Ru0.js` (gebaut, 1 Datei):

	Vorher (Dev-Quelle)	Nachher (Build)
Anzahl Dateien	13 einzelne <code>.js</code> -Dateien	1 einzige Datei
Größe (gesamt)	51.197 Bytes	23.269 Bytes (-54 %)
Zeilen	z. B. <code>js/main.js</code> 92 Zeilen, <code>js/data.js</code> 111 Zeilen — lesbar formatiert	0 Zeilenumbrüche — der komplette Inhalt steht auf einer Zeile

	Vorher (Dev-Quelle)	Nachher (Build)
Variablennamen	sprechend: <code>state</code> , <code>caseRes</code> , <code>loadCorePeopleAndLocations</code>	eingedampft auf <code>e</code> , <code>t</code> , <code>n</code> , ...
Kommentare	alle unsere UE1/UE2- Erklärkommentare	komplett entfernt
Dateiname	<code>main.js</code> , <code>data.js</code> , ... (fest)	<code>index-B1hg0Ru0.js</code> (Name hängt vom Inhalt ab)

`index.html` (Quelle vs. gebaut) — drei verschiedene Sorten von Änderung an einer Datei:

1. `<script type="module" src="js/main.js">` (Original: ganz am Ende von `<body>`) → im Build **nach** `<head>` **verschoben**, umgeschrieben zu `<script type="module" crossorigin src="/assets/index-B1hg0Ru0.js">`.
2. `<link rel="stylesheet" href="styles.css">` → ebenfalls in `<head>`, umgeschrieben zu `/assets/index-ZAWMSz9M.css`, `crossorigin` ergänzt.
3. `` (Logo) → **byte-identisch unverändert** — weil das eine `public/`-Datei ist, fasst Vite sie bewusst nicht an.

Genau dieser dritte Punkt ist die beste Live-Demonstration dafür, was `public/` bedeutet: **zwei** Referenzen in derselben `index.html`, eine wird komplett transformiert (Bundling+Hash), die andere bleibt exakt gleich — je nachdem, ob die Datei über `public/` läuft oder über den Modul-Graphen.

Verifikation

Voller Feature-Durchlauf über `vite preview` (nicht mehr `vite dev`, nicht mehr `python -m http.server`): alle 5 Views, Bookmark-Klick + `localStorage`-Eintrag als echter End-to-End-Beweis, Konsole leer, Netzwerk-Log zeigt nur noch die 3 gebauten Kern-Dateien + die unveränderten `public/`-Inhalte.

Live-Demo — was du im Unterricht herzeigst

1. Build live ausführen

```
npm run build
```

Zeig die Terminal-Ausgabe live — "17 modules transformed", die drei Zeilen mit Dateigröße + gzip-Größe, "built in 136ms". Sag: "Vite hat gerade 13 Dateien zu einer gemacht und alles verkleinert."

2. `dist/`-Ordner im Explorer/VS Code aufklappen

Zeig die Struktur: `index.html`, `assets/index-*.js`, `assets/index-*.css`, plus `data/` und `assets/logo/` `assets/people` unverändert. Sag den einen Satz: "Die Namen mit dem komischen Anhang sind neu gebaut, `data/` und die Bilder sind einfach 1:1 rüberkopiert."

3. Den gebauten JS-Code aufmachen (`dist/assets/index-*.js`)

Kurz reinscrollen (oder Editor-Zeilenzähler zeigen: "0 Zeilenumbrüche, alles auf einer Zeile") — Kontrast zu `js/data.js` danebenlegen (111 lesbare Zeilen mit Kommentaren). Das ist der überzeugendste visuelle Beweis für Minifizierung.

4. `dist/index.html` neben der Quell- `index.html` aufmachen

Zeig die drei Änderungen: `<script>/</link>` jetzt in `<head>`, mit gehashtem Dateinamen; `` **exakt gleich** in beiden Dateien — "eine Referenz wurde umgebaut, die andere absichtlich nicht, weil sie aus `public/` kommt."

5. Preview starten, DevTools → Network

```
npm run preview
```

`http://localhost:4173` öffnen, `F12` → Network-Tab, Seite laden. Zeig: nur noch 3 eigene Dateien (JS, CSS, Logo) + die `data/ / assets`-Dateien aus `public/` — **kein** `@vite/client`, **kein** `node_modules`-Request mehr. Sag: "das ist exakt das, was später online steht, keine Entwickler-Extras mehr dabei."

6. Interaktivität live beweisen

Ein Beweisstück bookmarken (Stern anklicken), dann DevTools → Application/Storage → Local Storage öffnen, den Eintrag `remotion_bookmarks` zeigen — "das läuft alles im minimierten Code genauso wie vorher, nur kleiner."

Demo 4 — `package.json`-Scripts: Lint & Format

Einfach erklärt — worum geht's hier überhaupt?

Bis jetzt hat niemand geprüft, ob unser Code eigentlich **gut** ist — nur ob er läuft. Demo 4 bringt zwei neue Helfer rein, die das automatisch für uns tun. Wichtig: das sind **zwei verschiedene Jobs**, die man leicht verwechselt.

Der Linter (bei uns: ESLint) ist ein Korrekturleser für die LOGIK. Er liest deinen Code und sucht nach Dingen, die vermutlich ein echter **Fehler** sind — eine Variable, die du deklarierst und nie benutzt (meistens ein Zeichen, dass du was vergessen hast), ein Vergleich, der nie wahr sein kann, etc. Ihm ist völlig egal, wie dein Code **aussieht** (Leerzeichen, Anführungszeichen).

Der Formatter (bei uns: Prettier) ist ein Layouter fürs AUSSEHEN. Er ändert **niemals**, was dein Code tut — nur wie er aussieht: Einrückung, Leerzeichen, Anführungszeichen, wann eine Zeile umgebrochen wird. Komplett automatisch, komplett konsistent, im ganzen Projekt gleich.

Analogie: stell dir ein Buchmanuskript vor. Ein **Lektor** (Linter) liest es und sagt "dieser Satz ergibt keinen Sinn, den musst du ändern" — das betrifft den **Inhalt**. Ein **Schriftsetzer** (Formatter) sorgt dafür, dass jeder Absatz gleich eingerückt ist, dieselbe Schriftgröße hat — das betrifft nur die **Optik**, der Text bleibt wortwörtlich derselbe. Beide machen das Buch besser, aber aus komplett unterschiedlichen Gründen — und sie widersprechen sich nicht, weil wir dem Lektor (`eslint-config-prettier`) explizit gesagt haben: "um Optik kümmert sich der Setzer, misch dich da nicht ein."

Warum zwei getrennte Befehle, `lint` und `lint:fix`? Nicht jeder gefundene Fehler ist sicher automatisch reparierbar. Konkretes Beispiel, das wir gleich selbst ausgelöst haben: eine ungenutzte Variable. ESLint **weigert sich**, die einfach zu löschen — es kann nicht wissen, *ob* das ein Tippfehler ist (dann löschen, sicher) oder ob du eigentlich **vergessen** hast, die Variable irgendwo zu benutzen (dann wäre Löschen ein neuer Bug!). Deshalb meldet `lint` es nur

und lässt **dich** entscheiden. `lint:fix` wendet nur die Korrekturen an, bei denen ESLint sich zu 100 % sicher ist, dass nichts kaputtgeht.

Was passiert bei `npm run lint` "unter der Haube"? npm schaut in `package.json` unter `scripts.lint` nach, findet dort den Text `"eslint ."`, und sucht dann nach einem Programm namens `eslint`, das es ausführen kann. Es sucht dabei **zuerst** in einem versteckten Ordner namens `node_modules/.bin/` — genau dort legt npm automatisch eine kleine "Startdatei" für jedes installierte Werkzeug ab, das ein Kommandozeilen-Programm mitbringt (wie unser `eslint`).

Task 1 — ESLint + Prettier installiert und konfiguriert

Befehl (Nutzer): `npm install -D eslint @eslint/js eslint-config-prettier globals prettier` →
`devDependencies: eslint ^10.11.0, @eslint/js ^10.0.1, eslint-config-prettier ^10.1.8, globals ^17.12.0, prettier ^3.9.8`. Alle fünf als `-D` (DevDependency) — laufen nur beim Entwickeln, ihr Code landet nie im Browser der Besucher:innen (Demo-1-Prinzip).

`eslint.config.js` neu (das moderne „Flat Config“-Format):

- `js.configs.recommended` — ESLints eigenes Regel-Set gegen echte Logikfehler (`no-unused-vars`, `no-undef`, `no-unreachable`, `no-dupe-keys`, ...).
- `globals.browser` — sagt ESLint, dass Dinge wie `window`, `document`, `fetch`, `localStorage` ganz normal existieren (sonst würde es sie fälschlich als "unbekannte Variable" melden).
- `ignores: dist/, public/, node_modules/, app.js, ue1/, ue2/`. `app.js` bewusst dabei — die eingefrorene Vor-Refactor-Datei aus UE1 wird nie mehr bearbeitet, sie zu linten würde nur jede Menge irrelevanter, längst bekannter Alt-Probleme melden.
- `eslint-config-prettier` ganz am Ende — schaltet die paar ESLint-eigenen Optik-Regeln ab, die sich mit Prettier streiten könnten.

`.prettierrc.json` — drei bewusste Einstellungen (nicht blind Standard übernommen):

- `"printwidth": 100` (Standard wäre 80) — unser Code hat viele lange HTML-String-Bauzeilen (`html += '<div class="...">' + ev.title + ...`), bei 80 würde Prettier die noch aggressiver umbrechen, als nötig ist.
- `"singleQuote": false` — passt zu unserem bisherigen Stil (doppelte Anführungszeichen außen).
- `"trailingComma": "es5"` — Kommas nach dem letzten Element in mehrzeiligen Listen/Imports, wo das gültige JavaScript ist (macht künftige Diffs kleiner: eine neue Zeile hinzufügen ändert dann nicht auch noch die Zeile davor).

`.prettierignore` — dieselbe Logik wie `.eslintignore`, plus `package-lock.json` (wird von npm selbst verwaltet, ein Formatter sollte das nie anfassen).

`package.json`: drei neue Scripts ergänzt: `"lint": "eslint .", "lint:fix": "eslint . --fix", "format": "prettier --write ."`.

Task 2 — `lint` fängt einen echten Fehler

Absichtlich eine ungenutzte Variable in `js/views/dashboard.js` eingebaut:

```
const unreviewedCount = 0;
```

`npm run lint` meldet:

```
D:\...\js\views\dashboard.js
  12:9  error  'unreviewedCount' is assigned a value but never used  no-unused-
vars

✖ 1 problem (1 error, 0 warnings)
```

Danach die Zeile **auskommentiert** (nicht gelöscht) und mit einem Erklär-Kommentar versehen, damit sie sich für die Live-Präsentation mit einem Klick reaktivieren lässt. `npm run lint` danach wieder **0 Fehler** — bestätigt, dass genau diese eine Zeile der Auslöser war.

Task 3 — `format` ändert echte Dateien

`npm run format` (`prettier --write .`) auf den kompletten, bisher nie formatierten Code losgelassen. Ergebnis laut `git diff --stat`:

```
13 files changed, 1610 insertions(+), 311 deletions(-)
```

Drei kleine, gut lesbare Beispiele aus dem echten Diff:

`.vscode/settings.json` — 4-Leerzeichen-Einrückung → 2, fehlender Zeilenumbruch am Dateiende ergänzt:

```
{
-   "liveServer.settings.port": 5501
-}
+ "liveServer.settings.port": 5501
+}
```

`js/utils.js` — eine zu lange Zeile (über `printwidth: 100`) wird umgebrochen und in Klammern gesetzt:

```
- return d.toLocaleDateString(undefined, { year: "numeric", month: "short", day:
"numeric" }) +
-   " " + d.toLocaleTimeString(undefined, { hour: "2-digit", minute: "2-digit"
});
+ return (
+   d.toLocaleDateString(undefined, { year: "numeric", month: "short", day:
"numeric" }) +
+   " " +
+   d.toLocaleTimeString(undefined, { hour: "2-digit", minute: "2-digit" })
+ );
```

`js/main.js` — Trailing Comma nach dem letzten Import-Namen ergänzt (unsere `trailingComma: "es5"`-Einstellung):

```
    closeEvidenceDetail,  
-   saveCurrentNote  
+   saveCurrentNote,  
  } from "./views/evidence.js";
```

`app.js` (in `.prettierignore`) blieb **komplett unangetastet** — `git status` zeigt keinerlei Änderung an dieser Datei, bestätigt die Ignore-Konfiguration.

Wichtig: **in keinem einzigen Diff ändert sich Logik** — nur Whitespace, Klammern, Kommas, Zeilenumbrüche. Direkt danach mit `npm run dev` + vollem Feature-Durchlauf im Browser bestätigt

(18 Evidenz-Karten, 6 Personen, 15 Timeline-Events, Dashboard-Stats, Sortierung) — die App verhält sich exakt wie vorher, nur der Quelltext sieht jetzt überall gleich aus.

Verifikation

`npm run lint` sauber (0 Fehler, der Test-Fall ist auskommentiert geparkt). `npm run format` läuft idempotent (ein zweiter Aufruf meldet für bereits formatierte Dateien `(unchanged)`). Voller Feature-Durchlauf über `npm run dev` nach dem Formatieren: alle 5 Views, Bookmark + Sortierung funktionieren, Konsole zeigt nur Vites eigene Verbindungs-Meldungen.

Live-Demo — was du im Unterricht herzeigst

1. `npm run lint` sauber zeigen

```
npm run lint
```

Keine Ausgabe = 0 Fehler. Sag: "unser Code ist aktuell sauber laut Linter."

2. Den geparkten Fehler live reaktivieren

In `js/views/dashboard.js` die Zeile `// const unreviewedCount = 0;` einkommentieren (das `//` weg), speichern. Dann nochmal:

```
npm run lint
```

Zeig die rote Fehlermeldung `'unreviewedCount' is assigned a value but never used`. Sag: "das ist eine Logik-Warnung, keine Optik-Frage — deshalb macht das der Linter, nicht der Formatter."

3. Zeigen, dass `lint:fix` das NICHT einfach wegräumt

```
npm run lint:fix
```

Der Fehler ist **immer noch da** — sag: "ESLint traut sich nicht, eine ungenutzte Variable einfach zu löschen, das könnte einen echten Bug verstecken. Das muss ein Mensch entscheiden." Danach die Zeile wieder auskommentieren, `npm run lint` zeigt wieder 0 Fehler.

4. Einen Formatierungs-Unterschied live erzeugen

In `styles.css` irgendwo bewusst unsauber einrücken (z. B. Tabs statt Leerzeichen, oder `color:red` ohne Leerzeichen nach dem Doppelpunkt) und speichern. Dann:

```
npm run format
```

Datei nochmal aufmachen — automatisch wieder sauber. Sag: "das hat mit Logik nichts zu tun, nur mit Aussehen — genau der Job des Formatters."

5. `git diff` zeigen (Statistik reicht)

```
git diff --stat
```

Zeig die Zeile `13 files changed, 1610 insertions(+), 311 deletions(-)` — "das war der einmalige große Aufräum-Diff, weil der Formatter den Code zum ersten Mal gesehen hat. Ab jetzt bleiben solche Diffs winzig, weil jeder neue Code gleich richtig formatiert reinkommt."

Demo 5 — TypeScript-Einstieg & erste Konvertierungen

Einfach erklärt — worum geht's hier überhaupt?

Was ist TypeScript? JavaScript + ein "Typ-Prüfer" obendrauf. Du schreibst fast denselben Code wie vorher, sagst dem Computer aber zusätzlich Dinge wie "das hier ist immer ein Text" oder "das hier könnte auch mal fehlen". Ein Programm (der TypeScript-**Compiler**, `tsc`) liest **bevor irgendwas läuft** deinen ganzen Code durch und meldet, wenn du dir selbst widersprichst — z. B. wenn du versprichst "das ist immer ein Text", aber an einer Stelle so tust, als wäre es eine Zahl.

Analogie: eine Rechtschreibprüfung, aber für **Logik statt Buchstaben**. Eine Rechtschreibprüfung führt deinen Aufsatz nicht aus, um Fehler zu finden — sie liest den Text und markiert Probleme, **bevor** du ihn abgibst. Genauso liest TypeScript deinen Code und markiert Typ-Widersprüche, **bevor** er je im Browser läuft.

Unterschied zu ESLint (Demo 4)? ESLint sucht nach **verdächtigen Mustern** ("diese Variable hast du nie benutzt"). TypeScript prüft etwas Grundlegenderes: passt der **Typ** jedes Werts zu dem, was jede Stelle, die ihn benutzt, erwartet? "Diese Funktion verspricht, immer einen Text zurückzugeben, aber auf diesem Pfad könnte sie auch `undefined` liefern" — das ist kein Stilfehler, das ist ein beweisbarer Widerspruch.

Die vier Schritte, die wir gemacht haben:

1. TypeScript installiert.
2. `tsconfig.json` geschrieben — TypeScripts eigene Einstellungsdatei (wie `eslint.config.js` aus Demo 4, nur fürs Typprüfen).
3. Zwei Dateien von `.js` zu `.ts` umbenannt und mit Typen versehen.

4. `tsc --noEmit` (nur prüfen, nichts erzeugen — Vite übersetzt weiterhin den eigentlichen JS-Code) in `npm run build` eingehängt.

Der spannendste Teil — ein Umweg, der genau das Richtige gezeigt hat: Beim ersten `npm run`

`typecheck` meldete TypeScript 7 Fehler in `lookup.ts`, obwohl der Code 1:1 aus dem funktionierenden

JavaScript übernommen war. Grund: `state.js` selbst ist noch **nicht** typisiert. TypeScript hat versucht zu erraten, was in `state.allEvidence` drinsteckt, hat dort im für TypeScript sichtbaren Code nur `allEvidence: []` (eine leere Liste) gesehen — und **nirgends** eine Stelle, die je etwas hineinlegt (das passiert ja erst zur Laufzeit per `fetch()`, in Dateien, die TypeScript hier noch gar nicht anschaut). Also hat TypeScript — aus seiner Sicht völlig logisch — geschlossen: "diese Liste kann nur immer leer sein". Offensichtlich falsch, wir wissen ja, dass sie mit echten Daten gefüllt wird — aber TypeScript kann **nur über das urteilen, was es im Code sieht**, nie über das, was beim tatsächlichen Ausführen passiert. Das ist eine der wichtigsten Lektionen zu TypeScript überhaupt: es ist ein **statischer** Prüfer — es führt dein Programm nie selbst aus.

Der Fix, einfach gesagt: wir haben TypeScript genau an der einen Stelle, wo wir `state` benutzen, direkt gesagt: "vertrau mir, diese drei Listen enthalten wirklich [Typ]-Objekte" — eine kleine, gezielte Ansage, **nicht** der große "schalt einfach jede Prüfung ab"-Hammer (`any`). Wie wenn du einem sehr wörtlich denkenden Kollegen sagst "ich weiß, du siehst den Lieferwagen nicht, der jeden Morgen die Ware bringt, aber glaub mir, das Regal wird wirklich befüllt" — statt zu sagen "prüf bei diesem Regal einfach gar nichts mehr".

Die Nachfolge-Fehler, einfach gesagt: selbst danach hat TypeScript noch unseren altbekannten "durch die Liste per Index durchgehen"-Code (`liste[i]`) bemängelt — weil generell "gib mir Element Nummer 5" schiefgehen kann, wenn die Liste kürzer ist als gedacht (in JavaScript kein Fehler, einfach still `undefined`). Wir hatten TypeScript explizit gebeten, genau darauf zu achten (`noUncheckedIndexedAccess`, siehe Task 1) — viele Teams lassen das aus, weil es JEDEN Listenzugriff etwas umständlicher macht, wir haben es bewusst drin gelassen, weil unser Code genau dieses Zugriffsmuster oft benutzt und ein "Index war doch nicht da"-Fehler genau die Art Bug ist, die wir in UE1 (Demo 2–5) mühsam von Hand jagen mussten. Fix: den Wert einmal holen, prüfen, dass er wirklich da ist, dann benutzen — gleiches Ergebnis wie vorher, nur jetzt auch für TypeScript nachvollziehbar sicher.

Task 1 — TypeScript installiert, `tsconfig.json` bewusst konfiguriert

Befehl (Nutzer): `npm install -D typescript` → `devDependencies.typescript: "^7.0.2"`.

`tsconfig.json` — jede Einstellung mit eigenem Grund, nicht blind von irgendwo kopiert:

Einstellung	Wert	Warum
-------------	------	-------

Einstellung	Wert	Warum
<code>target / lib</code>	<code>ES2022,</code> <code>["ES2022",</code> <code>"DOM",</code> <code>"DOM.Iterable"]</code>	passt zu "aktuelle evergreen-Browser" (README); <code>DOM</code> nötig, weil wir direkt <code>document / window</code> anfassen
<code>moduleResolution</code>	<code>"Bundler"</code>	die Auflösungs-Strategie extra für bundler-basierte Tools wie Vite — nicht für direktes Node-Ausführen gedacht
<code>isolatedModules</code>	<code>true</code>	Vite übersetzt mit esbuild , das jede Datei einzel n übersetzt, ohne das ganze Programm zu kennen — dieses Flag lässt <code>tsc</code> genau die TS-Konstrukte melden, die dabei stillschweigend kaputtgehen würden
<code>noEmit</code>	<code>true</code>	<code>tsc</code> wird bei uns nur zum Prüfen benutzt — das eigentliche Übersetzen macht weiterhin Vite/esbuild
<code>allowJs + checkJs: false</code>	<code>true / false</code>	lässt <code>.ts</code> -Dateien <code>.js</code> -Dateien überhaupt importieren (<code>lookup.ts</code> → <code>state.js</code>), aber verlangt noch nicht , dass der Inhalt von <code>state.js</code> selbst fehlerfrei typisiert ist — das kommt in Demo 6/7
<code>strict</code>	<code>true</code>	die Haupt-Entscheidung dieser Demo — siehe F1
<code>noUncheckedIndexedAccess</code>	<code>true</code>	bewusst zusätzlich über <code>strict</code> hinaus an — siehe oben, hat direkt echte Stellen in <code>lookup.ts</code> gefunden
<code>noUnusedLocals /</code> <code>noUnusedParameters</code>	<code>false</code>	bewusst aus — das deckt ESLint (Demo 4) schon ab, zwei Werkzeuge mit derselben Meldung wäre nur doppelt laut

`include: ["js/**/*ts"]` — das Typ-Programm besteht aktuell nur aus unseren zwei konvertierten

Dateien (plus, über `allowJs`, dem noch ungeprüften `state.js`, das sie importieren). Der Rest der

App (`main.js`, alle Views, `app.js`) ist für `tsc` aktuell schlicht nicht sichtbar — das ändert sich erst in Demo 7.

Task 2 — zwei Dateien konvertiert, ohne `any`, 0 Fehler

`js/utils.ts` (per `git mv`, Historie bleibt): komplett eigenständig, keine Imports — der einfachstmögliche erste Schritt. Parameter wie `formatDate` `s` `ts` bekommen den Typ `string | undefined | null`, weil genau das ist, was `ev.timestamp/evt.time` aus den JSON-Daten tatsächlich sein können.

`js/lookup.ts`: fünf Funktionen, drei kleine lokale Interfaces (`EvidenceRecord`, `PersonRecord`, `LocationRecord` — nur mit den Feldern, die diese Funktionen wirklich anfassen, nicht die vollständigen Domain-Typen, die kommen in Demo 6). Dazu der gezielte `stateJs` `as` `AppState`-Type-Assertion (siehe oben) und die `noUncheckedIndexedAccess`-Fixes in allen drei `findBy`-Funktionen + `countEvidenceForPerson`.

`grep -n '\bany\b'` auf beiden Dateien: `any` kommt **nur** in Erklär-Kommentaren vor, nirgends als echter Typ.

Task 3 — TypeScript ins Tooling eingehängt

`package.json`:

```
"typecheck": "tsc --noEmit",  
"build": "tsc --noEmit && vite build",
```

`build` bricht jetzt **vor** `vite build` ab, wenn `tsc` einen Fehler findet (`&&` verkettet die beiden Befehle — der zweite läuft nur, wenn der erste mit Erfolg endet). Live geprüft: mit einem absichtlichen Fehler (`const x: string = 5;`) bricht `npm run build` schon bei `tsc` ab, „vite build“ taucht in der Ausgabe gar nicht erst auf.

Bewusst nicht gemacht: `npm run dev` (Vites Dev-Server) selbst typprüft **nicht** — Vite übersetzt `.ts`-Dateien beim Entwickeln nur mit esbuild (schnell, aber esbuild entfernt Typen einfach, ohne sie zu validieren). Das ist eine bewusste Vite-Design-Entscheidung für Geschwindigkeit, kein Versehen unsererseits. Für Echtzeit-Fehler beim Tippen sorgt der Editor (VS-Code + TypeScript-Sprachserver); für die **verlässliche** Prüfung, die wirklich niemand umgehen kann, sorgt jetzt `npm run build` (und später die CI in Demo 8/9).

Verifikation

`npm run typecheck` → 0 Fehler. `grep` bestätigt: kein echtes `any` in den zwei neuen Dateien. `npm run dev` + voller Feature-Durchlauf im Browser: 18 Evidenz-Karten, `formatDate` (`utils.ts`) formatiert Daten korrekt, People-Cross-Link (nutzt `countEvidenceForPerson` / `evidenceMentionsPerson` aus `lookup.ts`) liefert 3 Karten — beide konvertierten Dateien funktionieren zur Laufzeit identisch zu vorher. Netzwerk-Log bestätigt: `GET /js/utils.ts` und `GET /js/lookup.ts` laden mit `200`, **obwohl** andere Dateien sie weiterhin mit `.js`-Endung importieren (`from "../utils.js"`) — Vite löst das automatisch auf die echte `.ts`-Datei auf, ganz ohne dass wir einen einzigen Import in den restlichen `.js`-Dateien anfassen mussten. `npm run build` läuft grün (`tsc --noEmit && vite build`), Konsole beim Dev-Server sauber.

Live-Demo — was du im Unterricht herzeigst

1. `tsconfig.json` aufmachen

Zeig 2–3 Zeilen und ihre Begründung aus der Tabelle oben — besonders

`noUncheckedIndexedAccess`

(„bewusst zusätzlich an, hat direkt einen echten Fall gefunden“) und `noUnusedLocals: false`

(„bewusst aus, das deckt schon ESLint ab“).

2. Den echten Fehler nachstellen

In `js/lookup.ts`, in einer der `findById`-Funktionen, `const ev = state.allEvidence[i]; + if (ev && ...) kurz zurück auf die alte Form ändern (if (state.allEvidence[i].id === id) return state.allEvidence[i];), speichern, dann:`

```
npm run typecheck
```

Zeig die roten `TS2532 / TS2322`-Fehler live. Erklär in einem Satz: "TypeScript kann nicht beweisen, dass der Index immer gültig ist." Rückgängig machen, `npm run typecheck` wieder grün.

3. Den Build-Gate-Test

In `js/utils.ts` die Zeile `// const kaputterTest: string = 5;` einkommentieren, dann:

```
npm run build
```

Zeig, dass die Ausgabe schon bei `tsc --noEmit` abbricht — "**vite v8.3.0 building...**" taucht gar nicht erst auf. Sag: "das ist der Beweis, dass Typfehler nicht nur im Editor rot sind, sondern den echten Build-Prozess blockieren." Zeile wieder auskommentieren, `npm run build` läuft wieder grün durch.

4. Netzwerk-Beweis für die `.js` → `.ts`-Auflösung

`npm run dev`, `F12` → Network-Tab, Seite laden. Zeig `GET /js/utils.ts` und `GET /js/lookup.ts`

mit `200` — "andere Dateien importieren die immer noch mit `.js` am Ende, Vite biegt das beim Laden automatisch auf die echte `.ts`-Datei um."

Demo 6 — Die Domain-Daten typisieren

(Hinweis: Demo 6/7 werden nicht präsentiert/angekreuzt — der Code muss trotzdem sauber funktionieren, weil Demo 8/9s CI genau darauf aufbaut. Diese Doku ist deshalb etwas knapper gehalten als bei den Demos, die wirklich vorgeführt werden.)

Task 1 — Typen für Evidence, Person, Location, Timeline-Event, Case

Neue Datei `js/types.ts` mit den Domain-Interfaces, passend zur echten Form von `data/*.json`:

`Evidence`, `Person`, `Location`, `TimelineEvent`, `CaseFile`, dazu drei Union-Typen

(`ReviewStatus`, `Relevance`, `Certainty` — nur mit `type` möglich, nicht mit `interface`, siehe

F3) und `AppStateShape` (die Zwischenlösung für den noch nicht typisierten `state.js`, siehe Demo 5).

Task 2 — Daten-Lader konvertiert

`js/data.js` → `js/data.ts` (per `git mv`). Die drei `fetch(...).then(res => res.json())`-Stellen

lesen jetzt nicht mehr stillschweigend `any`, sondern werden explizit zugeordnet:

```
state.caseData = (await caseRes.json()) as CaseFile;
state.allPeople = (await peopleRes.json()) as Person[];
state.allLocations = (await locationsRes.json()) as Location[];
state.allEvidence = (await res.json()) as Evidence[]; // in loadEvidenceData
state.allTimeline = (await res.json()) as TimelineEvent[]; // in
loadTimelineData
```

Zwei echte Compiler-Funde unterwegs:

1. Derselbe `never[]`-Effekt wie in Demo 5 (`state.js`'s leere Array-Literale) — behoben mit demselben Muster: `stateJs as AppStateShape` an der einen Importstelle.
2. **Neu:** `state.js` `caseData: {}` (leeres Objekt als Startwert) und unser `CaseFile` (9 Pflichtfelder) waren TypeScript laut eigener Meldung *"zu unähnlich"* für eine direkte Umwandlung (*"may be a mistake because neither type sufficiently overlaps with the other"*). Lösung: der von TypeScript selbst vorgeschlagene Umweg über `unknown` — `stateJs as unknown as AppStateShape`. Wichtig: das ist **kein** `any`. Es ist eine einmalige, bewusste Umwandlung an **einer** Stelle; jede Codezeile danach (`state.allEvidence.length` usw.) wird weiterhin ganz normal gegen die echten Typen geprüft.

`lookup.ts` gleichzeitig aufgeräumt: die Demo-5-Platzhalter `EvidenceRecord / PersonRecord / LocationRecord` (nur die paar Felder, die die Funktionen damals brauchten) sind jetzt durch die echten, vollständigen `Evidence / Person / Location`-Typen aus `types.ts` ersetzt.

Task 3 — das ambige Feld: `Evidence.personIds`

Direkt am Feld in `types.ts` dokumentiert: `personIds` ist fast überall eine echte `Person.id` (z. B. `"kernel-co1t"`) — **außer** bei E04 in `evidence.json`, wo stattdessen der **Anzeigename** `"Nova Byte"` drinsteht statt der id `"nova-byte"`.

Wie JavaScript das nie entscheiden musste: `personIds` war einfach `Array` — ein Array aus Strings ist ein Array aus Strings, ob eine id oder ein Name drinsteht, ist JS völlig egal. `evidenceMentionsPerson()` (schon aus UE1 bekannt) prüft deshalb explizit **beides** (`indexOf(person.id) !== -1 || indexOf(person.name) !== -1`) — eine stille Notlösung, tief in einer Funktion vergraben, nie irgendwo als offizielle Aussage festgehalten.

Was das Schreiben eines ehrlichen Typs erzwungen hat: TypeScript selbst hat diese Inkonsistenz **nicht automatisch erkannt** (`string[]` ist für eine id genauso gültig wie für einen Namen — der Compiler kann den Unterschied nicht sehen). Was TypeScript stattdessen erzwungen hat: der **Versuch**, ein ehrliches Interface zu schreiben, hat mich gezwungen, mir die echten Daten nochmal genau anzuschauen und mich zu fragen "ist das *wirklich* immer eine id?" — und die

Antwort ("nein, siehe E04") jetzt explizit als Kommentar direkt am Typ festzuhalten, statt sie wie bisher nur implizit in einer Workaround-Funktion zu verstecken.

Verifikation

`npm run typecheck` → 0 Fehler (nach den zwei oben beschriebenen Funden behoben). `grep` bestätigt:

kein echtes `any` in `data.ts / lookup.ts / types.ts`. `npm run dev` + Feature-Durchlauf:

Dashboard

zeigt Case-Titel korrekt (`caseData` jetzt als `CaseFile` typisiert), alle 18/6/15 Einträge laden,

Konsole sauber. `npm run build` läuft grün (`tsc --noEmit && vite build`), Bundle-Größe

praktisch

unverändert (~23 KB).

Demo 7 — Die komplette App migrieren

(Hinweis: Demo 6/7 werden nicht präsentiert/angekreuzt — der Code muss trotzdem sauber funktionieren, weil Demo 8/9s CI genau darauf aufbaut. Diese Doku ist deshalb etwas knapper gehalten als bei den Demos, die wirklich vorgeführt werden.)

Alle 10 verbliebenen `.js`-Module (`main`, `navigation`, `data`, `storage`, `state`, `utils`, `dropdowns`, sowie `views/dashboard`, `views/evidence`, `views/people`, `views/timeline`, `views/workspace`) auf `.ts` umgestellt (per `git mv`). Zusammen mit den bereits vorhandenen `types.ts / utils.ts / lookup.ts / data.ts` (Demo 5/6) und dem neuen `dom.ts` besteht die App jetzt

komplett aus TypeScript-Dateien. `npm run typecheck` (das ganze Programm als ein zusammenhängendes

Ganzes, nicht mehr nur einzelne, isoliert geprüfte Dateien) läuft mit **0 Fehlern**.

Task 1 — Rest-Migration, ganze App kompiliert mit 0 Fehlern

Reihenfolge der Konvertierung (jede Datei einzeln typisiert, `npm run typecheck` nach jeder größeren Datei zwischendurch laufen lassen, um Fehler dort zu fixen, wo sie entstanden sind):

1. `js/dom.ts` (neu) — ein kleiner, wiederverwendbarer Helfer:

```
export function requireElement<T extends HTMLElement = HTMLElement>(id:
string): T {
  const el = document.getElementById(id);
  if (!el) {
    throw new Error(`Expected #${id} to exist in the DOM`);
  }
  return el as T;
}
```

`getElementById` liefert laut Typ immer `HTMLElement | null` — jede `id` *könnte* fehlen. Bei uns

sind die IDs aber fest in `index.html` verdrahtet und garantiert da, sobald die jeweilige View rendert. Statt an jeder der ~40 Fundstellen im Code `as HTMLSelectElement` + `!` zu wiederholen,

bündelt `requireElement<T>()` das an einer Stelle — mit generischem `<T>`, damit man z. B.

- `requireElement<HTMLSelectElement>("filterType")` schreiben kann und direkt `.value` typsicher dran hat, statt danach nochmal selbst zu casten.
2. `js/state.ts` — bekommt jetzt sein eigenes vollständiges `AppState`-Interface (nutzt die echten Domain-Typen aus `types.ts`). Das macht den `as unknown as AppStateShape`-Umweg aus Demo 5/6 überflüssig: `state.js` selbst war die Lücke, die diesen Umweg nötig gemacht hat — `lookup.ts` und `data.ts` importieren `state` ab jetzt direkt, ohne jede Zusicherung. `caseData: {} as CaseFile` bleibt als dokumentierter Platzhalter-Wert stehen (echter Inhalt kommt erst nach dem ersten `fetch`), aus demselben Grund wie schon in Demo 5/6: eine gezielte, kommentierte Zusicherung statt einer invasiveren `CaseFile | null`-Änderung überall im Code.
 3. `js/storage.ts`, `js/dropdowns.ts`, `js/navigation.ts` — mechanisch: Rückgabetypen (`void`), Lese-Schleifen über `NodeList` en auf `for...of` umgestellt (siehe Task 2), `unknown` + Typ-Check für `JSON.parse`-Ergebnisse aus `localStorage` (gleiches Muster wie Demo 5.5s `try/catch`-Fix, nur jetzt auch typsicher: `const parsed: unknown = JSON.parse(raw); ... Array.isArray(parsed) ? (parsed as string[]) : []`).
 4. `js/views/dashboard.ts`, `js/views/people.ts`, `js/views/timeline.ts` — durchgängig `requireElement()` statt ungeguardeter `getElementById`-Aufrufe, Klick-Callbacks typisieren `event.target as HTMLElement`.
 5. `js/views/evidence.ts` (die größte Datei, 388 Zeilen) — siehe Task 2 für die zwei nennenswerten Funde darin (`<select>`-Wert-Zusicherung, `closest<T>()`).
 6. `js/views/workspace.ts` — `HypothesisDraft`-Interface neu in `types.ts`, `Partial<HypothesisDraft>` beim Einlesen aus `localStorage` (siehe Task 2).
 7. `js/main.ts` (zuletzt, weil es alle anderen Module gleichzeitig importiert — der Test, ob wirklich alles zusammenpasst) — siehe Task 2 für den `window`-Fund.

Task 2 — mindestens drei Stellen, die echtes Nachdenken brauchten (nicht nur mechanisches Tippen von Typen)

1. `window.navigateTo = navigateTo`; (und 5 weitere) in `main.ts`. `index.html` hat noch inline `onclick="navigateTo('evidence')"`-Attribute aus der Zeit vor dem ES-Modul-Split (UE1) — die zugehörigen Funktionen müssen deshalb auf `window` liegen, damit HTML sie überhaupt findet. TypeScript's eingebautes `window`-Interface kennt diese sechs Custom-Properties aber naturgemäß nicht — `window.navigateTo = ...` wäre ohne Weiteres ein Fehler (Property 'navigateTo' does not exist on type 'window'). Die falsche, aber naheliegende Lösung wäre `(window as any).navigateTo = ...` gewesen. Stattdessen: TypeScript's eigener

Mechanismus für genau diesen Fall, **Declaration Merging** — ein bereits existierendes `interface` (hier: das globale `window`) erneut öffnen und um die eigenen Felder erweitern:

```
declare global {  
  interface Window {  
    navigateTo: (viewName: string) => void;  
    // ... 5 weitere  
  }  
}
```

Damit bleibt jede der sechs Zuweisungen weiterhin **strukturell geprüft** (falsche Parameteranzahl/-typen würden immer noch auffallen) — nur die *Existenz* dieser sechs zusätzlichen Felder auf `window` wird ehrlich zugesichert, statt jede Prüfung an dieser Stelle komplett abzuschalten.

2. `(e.target as HTMLSelectElement).value as ReviewStatus` in `views/evidence.ts` (`renderEvidenceDetail`). Ein `<select>`-Element liefert `.value` immer als reinen `string` —

das ist so in der DOM-Spezifikation festgelegt und kann TypeScript nicht enger machen. Unser

Domain-Modell (`Evidence.status: ReviewStatus`) ist aber die engere Union

`"unreviewed" | "reviewed" | "flagged"`. Eine blinde Zuweisung (`ev.status = e.target.value`)

wäre ein Typfehler — zu Recht, denn *irgendein* `<select>` könnte theoretisch jeden beliebigen

String als `value` haben. Die Zusicherung `as ReviewStatus` ist hier **bewusst gerechtfertigt** (nicht einfach der bequeme Weg um den Fehler herum), weil die drei `<option value="...">` für

genau dieses `<select>` von **unserem eigenen Code** erzeugt werden (`statusOptionHTML`, ein paar Zeilen weiter oben in derselben Datei) — der Wertebereich ist also tatsächlich garantiert, nur eben nicht auf eine Art, die TypeScript über zwei getrennte Funktionen hinweg selbst herleiten kann.

3. `noUncheckedIndexedAccess` **bei jedem** `NodeList`-**Loop**. Praktisch jede Schleife über `document.querySelectorAll(...)` stand im JS-Original als klassischer Index-Loop (`for (let i = 0; i < list.length; i++) { list[i]... }`). Mit

`noUncheckedIndexedAccess`

(Demo 5, bewusst zusätzlich zu `strict` an) ist `list[i]` vom Typ `T | undefined` — TypeScript

kann bei einem Index-Zugriff nie beweisen, dass er im gültigen Bereich liegt. Statt an jeder einzelnen Stelle `list[i]` + Null-Check zu schreiben (das Muster aus Demo 5/6 für `state.allEvidence[i]`, wo der Index selbst noch gebraucht wurde), war hier die bessere Lösung,

den **Index selbst wegzulassen**: `for (const item of list)` braucht

`noUncheckedIndexedAccess`

gar nicht erst zu berücksichtigen, weil nie indiziert wird. Betrifft u. a.

`navigation.ts` (`.view/ .nav-btn`-Listen), `evidence.ts`, `people.ts`, `timeline.ts`, `workspace.ts`. Kein Verhaltensunterschied, aber deutlich weniger Code pro Stelle als die capture-und-prüf-Variante.

Task 3 — Verifikation, kein `any`, App verhält sich unverändert

`grep -rn '\bany\b'` über den kompletten `js/`-Ordner: **kein einziger echter `any`-Typ** — die paar Treffer sind ausschließlich Erklär-Kommentare (`main.ts`: *warum* kein `any`; `data.ts`: dass `res.json()` von Natur aus `Promise<any>` ist). `find js -name "*.js"` findet **keine** Datei mehr — alle 15 Dateien unter `js/` sind `.ts` (die 10 Module aus dieser Demo + `dom.ts` / `types.ts` aus Demo 5–7 + `utils.ts` / `lookup.ts` / `data.ts` aus Demo 5/6).

`npm run typecheck` → **0 Fehler**, geprüft als das erste Mal, dass **alle** Module gleichzeitig gegeneinander typgeprüft werden (vorher war jede Datei mehr oder weniger isoliert dran).

`main.ts`

war der eigentliche Belastungstest dafür, weil es als einziges Modul praktisch alle anderen gleichzeitig importiert und benutzt.

`npm run dev` + voller Browser-Durchlauf (alle 5 Views, Suche/Filter/Sortierung, Bookmark setzen, Status/Relevance in der Detail-Ansicht ändern, Notiz speichern, Timeline-Filter + Quick-View-Modal,

Workspace-Hypothese ausfüllen/speichern/nach Reload prüfen): App verhält sich **identisch** zum Stand vor der Migration — reine Typ-Ergänzung, keine Logikänderung. `npm run build` (`tsc --noEmit && vite build`) läuft grün, 18 Module transformiert, Bundle ~21 KB (minimal kleiner

als vorher, weil im Zuge der Konvertierung auch ein paar tote Debug-Codezeilen aus früheren Demos

mit aufgeräumt wurden — siehe Kommentare in `main.ts`).

Demo 8 — GitHub Actions: Development-Workflow

Einfach erklärt — worum geht's hier überhaupt?

Bisher musste **ich mich selbst erinnern**, `npm run lint` / `npm run build` einzutippen, bevor ich etwas pushe. Ab jetzt übernimmt das ein Roboter: GitHub startet bei jedem `git push` automatisch

einen **leeren Computer** im Hintergrund, lädt den aktuellen Code drauf und führt dort Befehle aus.

Das ganze Prinzip heißt **CI (Continuous Integration)** — GitHubs Umsetzung davon heißt **GitHub Actions**, konfiguriert über eine YAML-Datei im Repo.

Die drei Begriffe, von außen nach innen:

Begriff	Was es ist	Bei uns konkret
Workflow	Die ganze Datei — legt fest, <i>wann</i> überhaupt etwas laufen soll	<code>.github/workflows/ci.yml</code> , <code>name: CI</code>
Job	Bekommt einen eigenen, komplett leeren Computer (virtuelle Maschine)	<code>lint-and-format:</code> — bei uns nur ein Job

Begriff	Was es ist	Bei uns konkret
Step	Ein einzelner Befehl innerhalb eines Jobs, läuft der Reihe nach auf derselben Maschine	5 Stück: Auschecken, Node einrichten, Installieren, Linten, Format prüfen

Analogie: der Workflow ist das ganze Rezeptbuch-Kapitel ("wann backe ich überhaupt"), ein Job ist ein einzelnes Rezept darin (bekommt seine eigene, saubere Küche), ein Step ist eine einzelne Anweisung im Rezept ("Ofen vorheizen", "Teig kneten") — die laufen der Reihe nach in derselben Küche und sehen, was die vorige Anweisung dort hinterlassen hat.

Warum überhaupt ein Roboter, wenn ich `lint` / `build` doch auch selbst laufen lassen kann? Weil

"könnte ich" nicht "tue ich immer" heißt — vergessen, Zeitdruck, oder schlicht ein Tippfehler im Terminal. CI ist die eine Stelle, die **niemand überspringen kann**, weil sie nicht auf deinem Rechner, sondern bei GitHub selbst läuft, bei jedem Push, auf einer immer gleichen, sauberen Maschine. Mehr dazu bei Frage 2 unten.

Die Workflow-Datei

```
name: CI

on:
  push:
  pull_request:
    branches: [main]

permissions:
  contents: read

jobs:
  lint-and-format:
    runs-on: ubuntu-latest
    steps:
      - name: Repo auschecken
        uses: actions/checkout@v4

      - name: Node.js einrichten
        uses: actions/setup-node@v4
        with:
          node-version: "24"
          cache: "npm"

      - name: Abhaengigkeiten installieren
        run: npm ci

      - name: Linter laufen lassen
        run: npm run lint

      - name: Formatierung pruefen (nur pruefen, aendert nichts)
        run: npm run format:check
```

Was jede Zeile bedeutet:

Codezeile	Erklärung
<code>on: push:</code>	Läuft bei jedem Push, auf jeden Branch — Probleme fallen sofort auf, egal wo gerade gearbeitet wird
<code>on: pull_request: branches: [main]</code>	Läuft zusätzlich bei jedem Pull Request gegen <code>main</code> — prüft Beiträge, bevor sie gemergt werden dürfen
<code>permissions: contents: read</code>	Der Roboter darf das Repo nur lesen . Dieser Workflow deployt nichts, braucht also keine Schreibrechte (bewusst so eng wie möglich — der Deploy-Workflow aus Demo 9/10 bekommt seine zusätzlichen Rechte dort explizit , nicht "auf Vorrat" schon hier)
<code>runs-on: ubuntu-latest</code>	Der leere Linux-Computer, den GitHub für diesen Job hochfährt
<code>uses: actions/checkout@v4</code>	Fertiges, von GitHub gepflegtes Mini-Programm ("Action"): holt das Repo per <code>git clone</code> auf den leeren Computer — der hat von sich aus keinen Code
<code>uses: actions/setup- node@v4, node-version: "24"</code>	Installiert Node.js — exakt dieselbe Major-Version wie lokal (<code>node --version</code> → <code>v24.14.1</code>), damit CI dieselben Ergebnisse liefert wie die eigene Maschine
<code>cache: "npm"</code>	Speichert heruntergeladene npm-Pakete zwischen zwei Läufen, siehe Tabelle unten
<code>run: npm ci</code>	Installiert exakt das aus <code>package-lock.json</code> , kein Neu-Auflösen von <code>^</code> -Versionsbereichen — strenger als <code>npm install</code> , siehe Frage 2 unten
<code>run: npm run lint</code>	Unser ESLint-Script aus Demo 4
<code>run: npm run format:check</code>	Neues Script (siehe unten) — prüft Formatierung, ohne etwas zu verändern

Was macht `cache: "npm"` genau?

Ohne Cache	Mit Cache
Jeder Lauf startet auf einer komplett leeren Maschine — jedes einzelne Paket wird bei jedem Push neu aus dem npm-Registry heruntergeladen	GitHub hebt den Download-Ordner (<code>~/ .npm</code>) nach dem Lauf auf und legt ihn beim nächsten Lauf automatisch wieder rein
Langsamer, unnötige Netzwerklast bei jedem einzelnen Push	Schneller — der Cache "verfällt" automatisch, sobald sich <code>package-lock.json</code> ändert (GitHub berechnet dafür selbst einen Hash der Datei als Cache-Schlüssel)

Ohne Cache	Mit Cache
Korrektheit identisch in beiden Fällen	<code>npm ci</code> hält sich so oder so exakt an die Lockfile — Cache betrifft ausschließlich Geschwindigkeit, nie das Ergebnis

Task 1 — Workflow geschrieben (Trigger, Checkout, Node-Setup mit Caching, Lint + Format-Check)

`.github/workflows/ci.yml` neu angelegt (Inhalt oben). Dazu in `package.json` ein neues Script ergänzt:

Datei	Änderung	Warum
<code>package.json</code>	<pre>"format:check": "prettier -- check ." neu</pre>	<code>format</code> (mit <code>--write</code>) verändert Dateien — das darf ein CI-Roboter nie von selbst tun, sonst wäre die Änderung nach dem Lauf sofort wieder weg, ohne dass sie je committet wurde. <code>--check</code> meldet nur (Exit-Code $\neq 0$ bei Abweichung), exakt dieselbe "melden statt heimlich reparieren"-Logik wie <code>lint</code> ohne <code>:fix</code> aus Demo 4

Ein wichtiger Fund dabei: `npm run lint` meldet aktuell **0 Probleme** — nicht weil der Code perfekt ist, sondern weil ESLint (Demo 4) nur `**/*.js`-Dateien prüft (`app.js` ist zusätzlich explizit ignoriert), und seit Demo 7 ist `js/` komplett `.ts`. ESLint prüft also gerade effektiv **gar nichts** mehr — kein Parse-Fehler, keine Meldung, einfach stille Zustimmung zu nichts. Volle `.ts`-Unterstützung bräuchte `@typescript-eslint` (eigener Parser + Regelwerk) — bewusst **nicht** Teil dieser Übung (zusätzliche Abhängigkeit, die die Aufgabenstellung nicht verlangt). Für die "CI schlägt fehl"-Demo (Task 2) wird deshalb `format:check` statt `lint` verwendet — Prettier deckt `.ts`-Dateien bereits vollständig ab, und die Aufgabenstellung erlaubt ausdrücklich "fails lint **or** format".

Task 2 — ein Commit, der bewusst fehlschlägt

Geplanter, minimaler Formatierungs-Verstoß: in `js/state.ts` bei `STORAGE_KEYS` testweise doppelte durch einfache Anführungszeichen ersetzt (`"remotion_bookmarks"` → `'remotion_bookmarks'`)

— verstößt gegen `.prettierrc.json`s `"singleQuote": false` (Demo 4), ändert aber **nichts** an der

Logik (gültiges TypeScript, `tsc`/ESLint sagen dazu nichts). Committet und gepusht — Ergebnis: der

Workflow schlägt beim Step „**Formatierung pruefen**“ fehl, die anderen vier Steps (Checkout, Node

einrichten, Installieren, Linter) bleiben grün. (wird als Nächstes live ausgeführt, Lauf-Link folgt)

Task 3 — Fix, erneuter Push, Workflow wieder grün

Anführungszeichen zurück auf doppelt (bzw. `npm run format` lokal laufen lassen), committet, gepusht. Ergebnis: derselbe Workflow läuft beim nächsten Push wieder komplett grün durch — alle

5 Steps bestanden. *(wird als Nächstes live ausgeführt, Lauf-Link folgt)*

Verifikation

(wird nach dem echten Push/Fail/Fix-Zyklus ergänzt: Links zu den drei Actions-Läufen — grün, rot, wieder grün)

Live-Demo — was du im Unterricht herzeigst

1. Die Datei zeigen

`.github/workflows/ci.yml` im Editor aufmachen. Einmal laut durchgehen: "ein Workflow, ein Job

(`lint-and-format`), fünf Steps." Auf `on:` zeigen: "läuft bei jedem Push und bei jedem PR gegen main."

2. Actions-Tab im Browser zeigen

GitHub-Repo → Tab **Actions**. Zeig den grünen Haken beim letzten Lauf, klick rein, zeig die 5 Steps einzeln aufklappbar mit ihren Logs.

3. Live einen Fehler auslösen

Im Editor: eine Anführungszeichen-Stelle irgendwo kurz auf einfache Quotes ändern, speichern.

```
git add -A
git commit -m "demo: format-verstoss fuer CI-vorfuehrung"
git push
```

Zurück zu GitHub Actions: neuer Lauf startet automatisch, Step "Formatierung pruefen" wird rot. Reinklicken, die Fehlermeldung von Prettier zeigen.

4. Live wieder reparieren

```
npm run format
git add -A
git commit -m "fix: Formatierung repariert"
git push
```

Neuer Lauf, wieder komplett grün. Sag den einen Satz: "das ist der Beweis — kein Mensch musste hier manuell draufschauen, der Roboter hat's von selbst gefangen und bestätigt."

Demo 9 — GitHub Actions: Deployment-Workflow

Einfach erklärt — worum geht's hier überhaupt?

Demo 8 hat nur **geprüft** (Linter, Formatierung) — nichts davon hat irgendwo etwas veröffentlicht. Demo 9 baut einen **zweiten, komplett separaten** Workflow, der die App wirklich baut und **live ins Internet stellt: GitHub Pages**, GitHubs eigenes kostenloses Hosting für statische Seiten. "Statisch" heißt: nur HTML/CSS/JS-Dateien, kein Server-Code, der irgendwas

berechnet — genau das, was `dist/` aus Demo 3 bereits ist.

Unterschied zu Demo 8, auf einen Blick:

	Demo 8 (<code>ci.yml</code>)	Demo 9 (<code>deploy.yml</code>)
Trigger	jeder Push, jeder PR gegen <code>main</code>	nur Push auf <code>main</code> + manueller Knopf
Tut was	prüft nur (Lint, Format)	baut (<code>vite build</code>) und veröffentlicht live
Braucht Rechte	nur lesen	lesen + veröffentlichen
Anzahl Jobs	1	2 (<code>build</code> → <code>deploy</code> , mit <code>needs: -</code> Abhängigkeit)

Warum ein zweiter Job, nicht einfach mehr Steps im selben Job? Weil "bauen" und "veröffentlichen" zwei fachlich verschiedene Dinge sind, die GitHub bewusst als zwei getrennte Phasen behandelt: `build` erzeugt ein **Artefakt** (eine gepackte Version von `dist/`, ähnlich einer ZIP-Datei, die GitHub selbst zwischenspeichert), `deploy` nimmt genau dieses Artefakt und macht es live. `needs: build` sagt: "starte `deploy` erst, wenn `build` fertig **und erfolgreich** war" — schlägt der Build fehl (z. B. ein Lint- oder TypeScript-Fehler), läuft `deploy` gar nicht erst los, die zuletzt live stehende Version bleibt unangetastet (siehe Demo 10, F1).

Zwei nötige Code-Änderungen

`vite.config.js` — `base: "/Mystery-Road-AWE-2026-lukesch/"` ergänzt:

Ohne <code>base</code>	Mit <code>base</code>
Vite schreibt alle Asset-Pfade im Build relativ zu <code>/</code> (z. B. <code>/assets/index-xxxx.js</code>)	Vite schreibt sie relativ zum Unterpfad (<code>/Mystery-Road-AWE-2026-lukesch/assets/index-xxxx.js</code>)
GitHub Pages liefert ein Projekt -Repo (kein User/Org-Pages-Repo) aber nicht unter <code>/</code> , sondern unter <code>/<repo-name>/</code> aus	Passt exakt zur echten URL: <code>https://plukesch.github.io/Mystery-Road-AWE-2026-lukesch/</code>
Ergebnis: 404 für jede JS-/CSS-/Bild-Datei, sobald die Seite live unter dem Unterpfad läuft — nur ein leeres <code><body></code> wäre sichtbar	App lädt korrekt

Betrifft **nicht nur** den Produktions-Build: Vite hängt `base` auch beim Dev-Server/Preview an die lokale URL an (`localhost:5173/Mystery-Road-AWE-2026-Lukesch/` statt `localhost:5173/`) — ein Aufruf von `localhost:5173/` allein leitet automatisch dorthin um, lokal ändert sich für den Arbeitsablauf sonst nichts.

`.github/workflows/deploy.yml` (neu) — der komplette Deploy-Workflow:

```
name: Deploy to GitHub Pages

on:
  push:
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read
  pages: write
  id-token: write

concurrency:
  group: pages
  cancel-in-progress: true

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Repo auschecken
        uses: actions/checkout@v4
      - name: Node.js einrichten
        uses: actions/setup-node@v4
        with:
          node-version: "24"
          cache: "npm"
      - name: Abhängigkeiten installieren
        run: npm ci
      - name: Linter laufen lassen
        run: npm run lint
      - name: Produktions-build
        run: npm run build
      - name: GitHub Pages vorbereiten
        uses: actions/configure-pages@v5
      - name: dist/ als deploybares Artefakt hochladen
        uses: actions/upload-pages-artifact@v3
        with:
          path: dist

  deploy:
    needs: build
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${ steps.deployment.outputs.page_url }
    steps:
      - name: Auf GitHub Pages veröffentlichen
```

```
id: deployment
uses: actions/deploy-pages@v4
```

Zeile für Zeile:

Codezeile	Erklärung
<code>on: push: branches: [main]</code>	Deployt nur , wenn wirklich auf <code>main</code> gepusht wird — nicht bei jedem Feature-Branch, nicht bei jedem PR (die werden nur geprüft, Demo 8)
<code>workflow_dispatch:</code>	Fügt einen manuellen "Run workflow" -Knopf im Actions-Tab hinzu — löst einen Lauf aus, ohne dass extra ein Dummy-Commit nötig ist (praktisch für die Live-Präsentation)
<code>permissions: contents: read</code>	Repo lesen — wie in Demo 8
<code>permissions: pages: write</code>	Erlaubt tatsächliches Veröffentlichen über die GitHub-Pages-API — der Dev-Workflow aus Demo 8 hatte das bewusst nicht
<code>permissions: id-token: write</code>	Erlaubt dem Workflow, sich per OIDC (ein kurzlebiger, automatisch ausgestellter digitaler Ausweis: "ich bin wirklich dieser eine Workflow-Lauf, gerade jetzt") gegenüber GitHub Pages auszuweisen — kein händisch verwaltetes Passwort/Secret nötig, siehe Demo 10 F2
<code>concurrency: group: pages</code>	Falls kurz hintereinander zwei Pushes passieren: der zweite Lauf wartet , statt gleichzeitig mit dem ersten um die Veröffentlichung zu konkurrieren
<code>jobs: build:</code>	Job 1 — baut die App, verpackt <code>dist/</code> als hochladbares Artefakt
<code>jobs: deploy: needs: build</code>	Job 2 — startet erst , wenn <code>build</code> fertig und erfolgreich war
<code>uses: actions/configure-pages@v5</code>	Bereitet die Pages-Konfiguration vor (liest u. a., unter welcher URL das Repo läuft)
<code>uses: actions/upload-pages-artifact@v3, path: dist</code>	Verpackt <code>dist/</code> als spezielles "Pages-Artefakt" — noch keine echte Veröffentlichung, nur ein Zwischenschritt, den <code>deploy</code> danach abholt
<code>environment: name: github-pages</code>	Ordnet den Job einer benannten GitHub- Environment zu — dort ließen sich später z. B. Freigabe-Regeln hinterlegen (nicht Teil dieser Übung, aber der vorgesehene Ort dafür)
<code>uses: actions/deploy-pages@v4</code>	Der eigentliche letzte Schritt: nimmt das hochgeladene Artefakt und macht es live unter der Pages-URL

Nötige manuelle Einstellung (kein Code, reine GitHub-Konfiguration)

GitHub Pages muss einmalig auf **Quelle: GitHub Actions** umgestellt werden (Standard ist "Deploy from a branch" — den älteren, hier nicht benutzten Weg über einen separaten `gh-pages` git-Branch, siehe Demo 9 F2). Repo → **Settings** → **Pages** (unter "Code, planning, and automation") → **"Build and deployment"** → **"Source"** → **"GitHub Actions"** auswählen.

Task 1 — zweiter Workflow: Checkout, Install, Lint, Build, Deploy auf GitHub Pages

`.github/workflows/deploy.yml` neu angelegt (Inhalt oben), `vite.config.js` um `base` ergänzt, GitHub-Pages-Quelle auf "GitHub Actions" umgestellt. Committet und gepusht.

Task 2 — deployte URL end-to-end bestätigt

`https://plukesch.github.io/Mystery-Road-AWE-2026-Tukesch/` im Browser geöffnet: App lädt vollständig (nicht nur eine leere Seite), Dashboard-Stats korrekt, alle 5 Views funktionieren, `data/*.json` und `assets/*` laden ohne 404 (der `base`-Fix greift). Nicht nur "Workflow zeigt grünen Haken", sondern die tatsächlich live erreichbare Seite geprüft — genau der Unterschied, den die Aufgabenstellung hier verlangt.

Task 3 — echte Änderung, Push, geht automatisch live

Eine kleine, sichtbare Änderung gemacht, committet und auf `main` gepusht — **kein** manueller Deploy-Schritt (kein Hochladen von Dateien per Hand, kein Knopf außer `git push`). Der `deploy.yml`-Workflow läuft automatisch an, baut neu, veröffentlicht die neue Version. Deployte URL danach erneut geöffnet: Änderung sichtbar live.

Verifikation

Workflow-Lauf im Actions-Tab: `build`-Job grün (Checkout → Node → Install → Lint → Build → Pages-Artefakt hochgeladen), danach `deploy`-Job grün (Veröffentlichung). Deployte URL im Browser end-to-end geprüft (siehe Task 2). Lokal weiterhin `npm run dev` / `npm run build` unverändert funktionsfähig (der `base`-Wert wirkt nur auf die tatsächlichen URLs, nicht auf die Funktionsfähigkeit selbst).

Live-Demo — was du im Unterricht herzeigst

1. Die deployte Seite zeigen

`https://plukesch.github.io/Mystery-Road-AWE-2026-Tukesch/` im Browser öffnen, kurz durch die Views klicken. Sag: "das ist keine lokale Vorschau mehr, das läuft öffentlich im Internet, GitHub selbst hostet das."

2. Die zwei Jobs im Actions-Tab zeigen

GitHub → Actions → letzter "Deploy to GitHub Pages"-Lauf. Zeig die zwei Kästchen `build` und `deploy` mit dem Pfeil dazwischen. Sag: "`deploy` startet erst, wenn `build` durch ist — steht hier in der Datei als `needs: build`."

3. Live einen echten Deploy auslösen

Im Editor irgendeine sichtbar kleine, harmlose Änderung machen (z. B. ein Wort im Dashboard-Text).

```
git add -A
git commit -m "demo: live-deploy vorfuehren"
git push
```

Zurück zu GitHub Actions: neuer Lauf startet automatisch (kein Klick nötig). Warten, bis beide Jobs grün sind, dann die deployte URL neu laden — Änderung ist live. Sag: "kein manueller Upload-Schritt, nur `git push`."

4. Alternative, falls gerade nichts zu ändern ist: manueller Trigger

Actions-Tab → Workflow "Deploy to GitHub Pages" → Button "**Run workflow**" (dank `workflow_dispatch:`) → löst denselben Ablauf ohne neuen Commit aus.

Demo 10 — Workflow-Trigger, Rechte & Fehlerfälle

Einfach erklärt — worum geht's hier überhaupt?

Demo 8 und 9 haben zwei funktionierende Workflows gebaut. Demo 10 fragt: **was, wenn etwas schiefgeht?** — passiert dann etwas Schlimmes mit der Live-Seite? Und: **wer darf hier überhaupt was?** — welche Rechte hat der Deploy-Roboter genau, und ist das riskant?

Task 1 — echter TypeScript-Fehler, blockiert Deploy real

In `js/utlils.ts` die geparkte Test-Zeile aus Demo 5 reaktiviert:

Schritt	Befehl/Änderung	Ergebnis
1	<code>// const kaputterTest: string = 5;</code> → <code>const kaputterTest: string = 5;</code> (Kommentar weg)	echter TS-Typfehler (<code>number</code> einer <code>string</code> -Variable zugewiesen)
2	<code>git add</code> , <code>git commit</code> , <code>git push</code>	löst beide Workflows aus
3	Ergebnis (echt beobachtet, Screenshot): Deploy to GitHub Pages / <code>build</code> X "Failing after 19s" — <code>deploy</code> -Job taucht gar nicht erst als Eintrag auf	<code>npm run build (tsc --noEmit && vite build)</code> bricht schon bei <code>tsc</code> ab, <code>deploy</code> startet nie, weil <code>needs: build</code> das verhindert
4	Rechts im GitHub-Sidebar unter "Deployments": die alte , zuvor erfolgreich deployte <code>github-pages</code> -Version blieb währenddessen weiterhin aktiv	direkter, echter Beweis für die Antwort zu Theorie-F1 unten

Schritt	Befehl/Änderung	Ergebnis
5	Zeile wieder auskommentiert, <code>git add / commit / push</code>	Ergebnis (echt beobachtet, Screenshot): alle 3 Checks grün (build 14s, deploy 1m, CI 10s), neues Deployment aktiv

CI (Demo 8) blieb bei Schritt 3 übrigens **grün** — der TS-Fehler ist kein Lint-/Format-Problem, ESLint und Prettier sagen dazu nichts (siehe Demo 7/8: ESLint deckt `.ts` ohnehin nicht ab). Nur der Build-Schritt (`tsc --noEmit`) fängt das — ein guter Beleg dafür, dass die drei Gates (Lint, Format, TypeScript) wirklich unterschiedliche Fehlerklassen abdecken, keine Doppelung.

Task 2 — Rechte & Secrets des Deploy-Workflows, wo konfiguriert

Was	Wert	Wo konfiguriert
<code>contents:</code> <code>read</code>	Repo lesen	<code>permissions:</code> -Block direkt in <code>.github/workflows/deploy.yml</code>
<code>pages:</code> <code>write</code>	Auf GitHub Pages veröffentlichen dürfen	ebenda
<code>id-token:</code> <code>write</code>	Sich per OIDC (kurzlebiger, automatisch ausgestellter Ausweis) gegenüber der Pages-API ausweisen	ebenda
GitHub-Pages-Quelle	"GitHub Actions" (statt "Deploy from a branch")	Repo → Settings → Pages → Build and deployment → Source (einmalig manuell umgestellt, Demo 9)
Secrets	keine	nicht nötig — die Authentifizierung läuft komplett über die drei <code>permissions:</code> -Zeilen + OIDC, kein von Hand angelegtes Passwort/Token in <code>Settings</code> → <code>secrets and variables</code> → <code>Actions</code>

Kein einziges Secret nötig, weil GitHub Pages als **GitHub-eigenes** Ziel direkt über den vom Workflow-Lauf selbst ausgestellten, kurzlebigen OIDC-Token authentifiziert — anders als bei einem externen Ziel (Netlify, eigener Server per SFTP, siehe Demo 9 F3), wo ein dauerhaftes API-Token/Passwort als Secret nötig wäre.

Task 3 — Run-Historie beider Workflows

Actions-Tab zeigt für **beide** Workflows getrennt die volle Lauf-Historie (`ci.yml` und `deploy.yml`, je eigener Menüpunkt in der linken Sidebar). Für die Präsentation: der Lauf aus Task 1 (roter Build, dann grüner Fix direkt danach) ist ein echter, vorzeigbarer Beleg — beim Reinklicken zeigt GitHub den genauen `tsc`-Fehlertext (Zeile, erwarteter vs. tatsächlicher Typ) direkt im Log des `build`-Jobs.

Verifikation

Beide Zyklen real durchgeführt und per Screenshot bestätigt (siehe Task 1). Keine offenen, kaputten Commits auf `main` — Endzustand ist der reparierte, grüne Stand.

Live-Demo — was du im Unterricht herzeigst

1. Den echten Fehlschlag reproduzieren

In `js/utis.ts` dieselbe Zeile wieder aktivieren (`// const kaputterTest...` → aktiv), pushen. Actions-Tab zeigen: `build` wird rot, `deploy` erscheint gar nicht. Reinklicken ins Log, die `tsc`-Fehlermeldung zeigen ("hier erklärt TypeScript live, warum eine Zahl keiner `string`-Variable zugewiesen werden darf").

2. Zeigen, dass die alte Seite live bleibt

Während (oder kurz nach) dem roten Lauf die deployte URL trotzdem öffnen — App läuft weiterhin, mit dem **alten** Stand. Sag: "ein kaputter Build nimmt die Seite nicht offline, er verhindert nur, dass eine neue, kaputte Version drüberkopiert wird."

3. Rechte zeigen

`.github/workflows/deploy.yml` aufmachen, auf den `permissions:`-Block zeigen. Dann GitHub →

Settings → Secrets and variables → Actions öffnen: **leer** — "wir brauchen kein einziges Secret, weil GitHub Pages sich direkt über diese drei Zeilen plus einen von GitHub selbst ausgestellten, kurzlebigen Token authentifiziert."

4. Reparieren, wieder grün zeigen

Zeile wieder auskommentieren, pushen, beide Workflows wieder grün zeigen, deployte URL neu laden.